



AL-HUSSEIN BIN TALAL UNIVERSITY

ANIMAL-DETECTION-SYSTEM (ADS)

TEAM MEMBERS:

HAMZA ALAMLEH

JMAL SALEH

ALHARETH ALTAWARH

ABD ALRAHMAN ALHAMAIL

YAZAN ALASMAR

AYHAM ALNAEEM

SUPERVISOR:

DR-RIYADH AL-SALAMIN

BACHELOR'S DEGREE

INFORMATION TECHNOLOGY

AL-HUSSEIN BIN TALAL UNIVERSITY

[2025/2026]

(CERTIFICATION OF PROJECT)

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

فَدَلَّاهُ لَدَامُنْ وَأَكْرَمَ الْفِتَاوَالْعَمَدِ جَا

صَدَقَ اللَّهُ الْعَظِيمُ

Permission to Use

In presenting this project in fulfilment of the requirements for a degree of Bachelor of Information Technology from Al-Hussein Bin Talal University, We agree that the University Library may make it freely available for inspection. We further agree that permission for the copying of this project in any manner, in whole or in part, for scholarly purpose may be granted by my supervisor(s) or, in their absence, by the College of Information Technology. It is understood that any copying or publication or use of this project or parts thereof for financial gain shall not be allowed without my written permission. It is also understood that due recognition shall be given to me and to Al-Hussein Bin Talal University for any scholarly use which may be made of any material from my project.

Requests for permission to copy or to make other use of materials in this project, in whole or in part, should be addressed to:

College of Information Technology
Al-Hussein Bin Talal University
Ma'an, Jordan

Abstract

Animal–vehicle collisions represent a major safety concern on roads that intersect wildlife habitats, posing serious risks to both human lives and animals. This project presents a low-cost, real-time Animal Detection and Alert System (ADS) designed to reduce such collisions using computer vision, edge computing, and cloud-based communication. The system employs a lightweight YOLOv5 object detection model trained on a custom dataset of commonly encountered animals, including dogs, cows, sheep, horses, and camels. The trained model achieved a mean Average Precision (mAP@0.5) of 83%, demonstrating reliable detection performance suitable for real-world deployment.

The detection system is implemented on a Raspberry Pi 5 equipped with a NoIR camera module, enabling continuous road monitoring under various lighting conditions. Upon detecting an animal, the system immediately alerts the driver through an onboard audio warning and transmits detection data, including GPS coordinates, to a cloud server via a SIM7600 4G cellular module. A collaborative alert mechanism is then activated using Firebase Cloud Functions, which identifies nearby vehicles within a one-kilometer radius and distributes real-time warnings through a dedicated mobile application.

By integrating real-time object detection, embedded hardware, cloud communication, and a mobile-based collaborative alert network, this project delivers an efficient, scalable, and cost-effective solution aimed at enhancing road safety and protecting wildlife, particularly in regions with limited infrastructure and frequent animal crossings.

Acknowledgement

We would like to express our sincere gratitude and appreciation to all those who contributed to the successful completion of this project.

First and foremost, we would like to thank our supervisor, Dr. Riyadh Al-Salamin, for his continuous guidance, valuable feedback, and unwavering support throughout all stages of this project. His technical expertise, constructive criticism, and encouragement played a crucial role in shaping the direction of the project and improving its overall quality.

We are also grateful to the College of Information Technology at Al-Hussein Bin Talal University for providing us with the academic environment, resources, and facilities necessary to carry out this work. The knowledge and skills we gained during our studies formed a strong foundation for the development of this project.

Special thanks go to our families for their patience, motivation, and moral support during the course of our studies and throughout the project duration. Their encouragement was a constant source of strength and determination.

Finally, we would like to acknowledge everyone who supported us directly or indirectly, whether through technical discussions, advice, or motivation. Their contributions, in one way or another, helped us achieve the successful completion of this project.

Table of Contents

CHAPTER 1 INTRODUCTION	1
1.1 Project Background.....	1
1.2 Problem Statement	2
1.3 Project Objectives	4
1.4 Project Significance	4
1.5 Project Gantt Chart	6
CHAPTER 2 LITERATURE REVIEW	7
2.1 Definitions.....	7
2.2 Object Detection	12
2.3 Collaborative Communication	18
2.4 The ADS Mobile Application: Architecture and Role in Collaborative Safety ..	23
2.5 Recent Work	27
CHAPTER 3 METHODOLOGY	29
3.1 Construct Framework	29
3.2 Develop System Architecture	29
3.3 Analyze and Design the System	30
3.4 Observe and Evaluate the System.....	31
3.5 Model Performance Evaluation	38
3.6 SIM7600 Operation	40
3.7 Publishing on and Receiving from Firebase	42
3.8 Explanation of the Code:.....	43
3.9 Cloud Integration with Firebase.....	45
3.10 Sequence Diagram of the Detection and Alert Process	47
3.11 Security Considerations	47
CHAPTER 4 ANALYSIS AND DESIGN.....	48
4.1 System Overview and Use Case Diagram	48
4.2 Data flow Diagram.....	49
4.3 Entity Relationship Diagram (ERD).....	51
4.4 Design Requirements.....	52
4.5 Engineering Standards	56

4.6 Realistic Constraints	58
4.7 Alternative and Different Design Approaches.....	61
4.8 Developed Design.....	62
CHAPTER 5 FINDINGS	69
5.1 Introduction.....	69
5.2 Project Interfaces and Their Description	69
5.3 Cloud-to-Application Interface (React-Based Monitoring Application).....	73
CHAPTER 6 OBSERVE AND EVALUATE	76
6.1 Raspberry Pi 5.....	76
6.2 Windows	81
6.3 Broadcast.....	84
CHAPTER 7 CONCLUSION	87
7.1 Key Strengths.....	87
7.2 Limitations	88
7.3 Future Work and Recommendations	88
7.4 Final Remarks	90

List of Table

Table 3-1 Number of images used in the final custom dataset.....	34
Table 3-2 : Table showing the parameters used for model training to obtain an optimal performance.....	36
Table 4-1 : Satisfaction of Requirements under Realistic Constraints	68
Table 6-1 : YOLOv5 Performance Metrics on COCO128 Dataset.....	76
Table 6-2 YOLOv5 Performance Metrics on COCO128 Dataset.....	77
Table 6-3 YOLOv5 Video Inference Summary on the Raspberry Pi 5.....	79
Table 6-4 Detection Frequency by Object Class.....	79
Table 6-5 YOLOv5 Frame-Wise Detection and Processing Summary on Raspberry Pi 5	80
Table 6-6 YOLOv5 Training Configuration and Detection Performance (Custom Animal Dataset).....	81
Table 6-7 YOLOv5 Inference Performance Comparison Across Image, Video, and Real-Time Scenarios (Windows Environment).....	83

Table 6-8 Broadcast Behavior Based on User 1 Detection Location	85
Table 6-9 YOLOv5 Performance Comparison on Raspberry Pi 5 vs windows	86

List of Figure

Figure 1-1 Gantt chart	6
Figure 2-1 Lane departure system and forward collision system operations [3].....	9
Figure 2-2 Vision camera monitoring [4].....	10
Figure 2-3 Night vision system operation in a car using an infrared sensor [5].....	11
Figure 2-4 The basic architecture design of two-stage object detectors [6]	12
Figure 2-5 The basic architecture design of one-stage detectors [7]	13
Figure 2-6 AIS01-LB LoRaWAN[18]	20
Figure 2-7 SIM7600E-H module ESP32-WROVER-B chip[19].....	21
Figure 2-8 Application project	23
Figure 2-9 From a project	24
Figure 2-10 From a project.....	24
Figure 3-1 representation of our 2-stage system	30
Figure 3-2 snippet of post-processing Class Filtering code	32
Figure 3-3 dataset setup on Roboflow	33
Figure 3-4 A graph showing YOLOv5 superior performance on AP and frames per second [30]	35
Figure 3-5 : snippet of the YAML file	37
Figure 3-6	41
Figure 3-7 Snippet of Wvdial configuration file	41
Figure 3-8	42
Figure 3-9 Snippet Python code to retrieve GPS data and publish it on Firebase.....	43
Figure 3-10 Subscription code snippet to receive messages from Firebase	44
Figure 3-11 Example Firebase Security Rules configuration.....	46
Figure 3-12	46
Figure 3-13 Snippet Cloud Functions code to filter sending alerts to devices are within1km	47
Figure 3-14 Sequence Diagram.....	47
Figure 4-1 Use case	49
Figure 4-2 Data Flow Diagram of the Animal Detection and Alert System	50
Figure 4-3 Entity Relationship Diagram (ERD) of the Animal Detection and Alert System .	52

Figure 4-4 : Flowchart for System Operation.....	63
Figure 4-5 System Hardware Components and Connectivity	65
Figure 4-6 System Architecture Diagram.....	67
Figure 6-1 YOLOv5 Inference Timing Breakdown	77
Figure 6-2 YOLOv5 Video Inference Summary on the Raspberry Pi 5	78
Figure 6-3 Detection Frequency by Object Class.....	78
Figure 6-4 YOLOv5 Detection Output on Video Frame (Raspberry Pi 5)	79
Figure 6-5 Test Images Using the Trained YOLOv5 Model.....	82
Figure 6-6 Test Video Using the Trained YOLOv5 Model	83
Figure 6-7 Test Real-Time Using the Trained YOLOv5 Model	83
Figure 6-8 User 2 receives the alarm.....	84
Figure 6-9 User 3 receives the alarm.....	84
Figure 6-10 Location Tracking Using SIM7600	85

CHAPTER 1

INTRODUCTION

In 2015, the General Directorate of Traffic in Saudi Arabia reported that stray camels on highways were responsible for nearly 400 deaths each year. These incidents also led to many injuries, long-term disabilities, and extensive vehicle damage. Motivated by the findings of this report, we began an in-depth investigation to identify effective solutions for reducing animal-vehicle collisions. [1] This project responds to these challenges by developing a comprehensive system that not only focuses on local detection but also extends to a collaborative alert network, where a custom-designed mobile application plays a pivotal role in delivering real-time warnings to drivers in nearby vehicles, thereby enhancing overall road safety.

1.1 Project Background

Animal–vehicle collisions (AVCs) have become an increasingly serious issue in many parts of the world, particularly in regions where roads cut through or border natural habitats, forests, and wildlife reserves. These incidents not only lead to the loss of wildlife but also endanger human lives, frequently resulting in significant vehicle damage, injuries, and even fatalities. In countries with high biodiversity or rapidly expanding road networks, the problem has worsened due to rising traffic volumes and habitat fragmentation caused by human development.

Conventional strategies for mitigating AVCs—such as warning signs, fencing, and wildlife overpasses—can be costly, limited in coverage, and often ineffective in real-time situations. With recent advances in computer vision and embedded systems, there is now a strong opportunity to create smarter, low-cost, and scalable solutions capable of detecting animals in real time and alerting drivers before collisions occur. These technologies hold considerable promise for reducing the prevalence of AVCs and supporting both public safety and wildlife conservation efforts.

This project addresses this challenge by developing a real-time animal detection and alert system utilizing a lightweight deep learning model, YOLOv5, optimized for deployment on

edge devices. The system is built around a Raspberry Pi 5 equipped with a Noir camera to continuously monitor the road ahead. When an animal is detected, the system triggers an immediate audio alert to the driver through a connected speaker and simultaneously sends detection data to a cloud server via a SIM7600 cellular module. The cloud platform then employs serverless computing (Firebase Cloud Functions functions) to disseminate alerts to nearby vehicles within a 1-kilometer radius, using periodically updated GPS coordinates to determine their proximity. This collaborative communication framework ensures that both the detecting driver and surrounding vehicles approaching the area receive timely warnings, thereby enhancing overall safety and system efficiency.

A key component that enables this collaborative network is a dedicated mobile application. This application serves as the critical interface for receiving and displaying cloud-relayed alerts to drivers in nearby vehicles. It operates seamlessly in the background, utilizing the vehicle's GPS to provide real-time location updates to the cloud server. When an alert is generated, the application instantly notifies the driver through clear visual and audible warnings, displaying essential information such as the type of animal detected and the approximate distance to the hazard. This integration of the mobile application completes the end-to-end safety ecosystem, transforming individual vehicles into interconnected nodes of a larger, more intelligent warning system.

By integrating real-time object detection, location-based alert distribution, and compact, low-power hardware, the system offers an affordable and practical solution for on-road animal detection. It is particularly beneficial in regions where interactions between wildlife and roadways are frequent and where large-scale monitoring infrastructure is lacking. Ultimately, this project contributes to the development of smarter, safer, and more environmentally responsible transportation systems.

1.2 Problem Statement

As road networks continue to extend into rural, natural, and forested regions, the number of animal–vehicle collisions (AVCs) has increased substantially, creating major risks for both human safety and wildlife preservation. These collisions frequently cause serious injuries, loss of life, and significant vehicle damage, particularly in regions with high wildlife movement. The World Health Organization (WHO) reports that road

traffic accidents are expected to become the fifth leading cause of death worldwide by 2030 [2]. Although accidents involving animals make up a smaller proportion of total road incidents, they still represent a notable concern. For example, the Jordan Traffic Police Report of 2020 shows that around 6% of road traffic accidents in the country were caused by animals crossing roadways—a percentage likely underestimated due to underreporting and insufficient monitoring systems. Despite these dangers, animal-related collisions have not received adequate attention in the development of modern road safety technologies.

In many regions, including Jordan, effective real-time tools to detect and respond to animal crossings are limited, especially at night or in remote areas where visibility is low and reaction times are shorter. Traditional countermeasures such as fixed signage or fencing lack adaptability and often fail to provide sufficient protection, particularly as road expansion continues to encroach on natural habitats. While advancements in autonomous systems and computer vision have made intelligent detection technologies possible, many of these solutions remain too costly, power-demanding, and infrastructure-dependent to be widely adopted in low- and middle-income regions. Additionally, the absence of real-time communication among nearby vehicles diminishes the effectiveness of individual detection units, preventing timely warnings that could help avoid multi-vehicle accidents.

Given these limitations, there is a pressing need for an affordable, energy-efficient, and scalable system is capable of detecting animals in real time, issuing immediate alerts to drivers, and distributing warnings to nearby vehicles through collaborative communication. This project addresses this need by developing an embedded, real-time animal detection and alert system that integrates edge AI with cloud-based communication. The system is designed not only to notify the driver in the detecting vehicle but also to broadcast alerts to surrounding vehicles within a 1-kilometer radius, thereby reducing collision risks, protecting wildlife, and improving overall road safety.

1.3 Project Objectives

- Design and develop a real-time animal detection and alert system aimed at enhancing driver awareness and significantly reducing animal–vehicle collisions (AVCs).
- Provide a cost-effective and scalable solution suitable for deployment in rural, forested, and wildlife-crossing areas where roads intersect with natural animal habitats.
- Utilize deep learning–based computer vision techniques to accurately detect animals near roadways in real time.
- Integrate embedded hardware platforms to enable on-site data processing and immediate system response.
- Employ cloud-enabled communication to share detection data and alerts efficiently between system components.
- Develop a dedicated mobile application that acts as the primary user interface for drivers.
- Deliver instant visual and audible alerts through the mobile application to warn drivers of nearby animal presence.
- Enable collaborative alert sharing between vehicles, allowing detected animal warnings to be communicated to nearby drivers in real time.
- Improve road safety for both humans and wildlife by minimizing collision risks and promoting proactive driver response.

1.4 Project Significance

Animal–vehicle collisions pose a significant risk to both roadway safety and wildlife conservation, particularly in regions where roads pass through natural habitats. These collisions commonly result in fatalities, injuries, vehicle damage, and the loss of wildlife, yet they remain insufficiently addressed in many areas. This project introduces a proactive and cost-effective solution through a real-time detection and alert system designed to substantially reduce such incidents. Unlike traditional mitigation measures—such as fencing and road signs—which are static and often limited in

effectiveness, the proposed system delivers automated, dynamic detection along with immediate driver warnings.

The system uses an affordable and compact hardware configuration, including a Raspberry Pi 5, Noir camera, speaker, and SIM7600 module, making it especially suitable for deployment in low- and middle-income countries where infrastructure and resources may be constrained. Powered by the YOLOv5 model, known for its high speed and dependable detection capabilities, the system achieves a mean Average Precision (mAP) of 83%. This level of performance allows for accurate real-time detection on resource-limited edge devices, giving drivers more time to react and avoid potential collisions.

A major innovation in this project is its collaborative communication component. By incorporating cloud services such as Firebase Cloud Functions, the system can identify vehicles within a 1-kilometer radius of the detection point and send them instant alerts. This creates a localized safety network that provides surrounding drivers with advance warnings of potential animal crossings, greatly improving reaction times and reducing the chances of multi-vehicle accidents. The dedicated mobile application is the crucial component that makes this network tangible to the end-user. It transforms the abstract cloud-based alert into an immediate, clear, and actionable warning directly on the driver's personal device, ensuring the safety message is received and acted upon.

Additionally, the system routinely updates the GPS coordinates of each participating vehicle to ensure accurate and timely alert distribution. This continuous location sharing, facilitated by the mobile application running in the background, is what enables the precise targeting of alerts only to vehicles in the relevant danger zone, maximizing efficiency and minimizing unnecessary distractions for other drivers. Beyond enhancing immediate road safety, this feature supports long-term tracking and data analysis, enabling refinements to the system and contributing valuable information for transportation planning and wildlife protection efforts.

In summary, this project delivers a scalable, efficient, and economically viable approach to a commonly overlooked issue. Its integration of AI, edge computing, and cloud-based communication enables a comprehensive solution for reducing animal-

vehicle collisions and promoting safer, more environmentally responsible transportation.

1.5 Project Gantt Chart

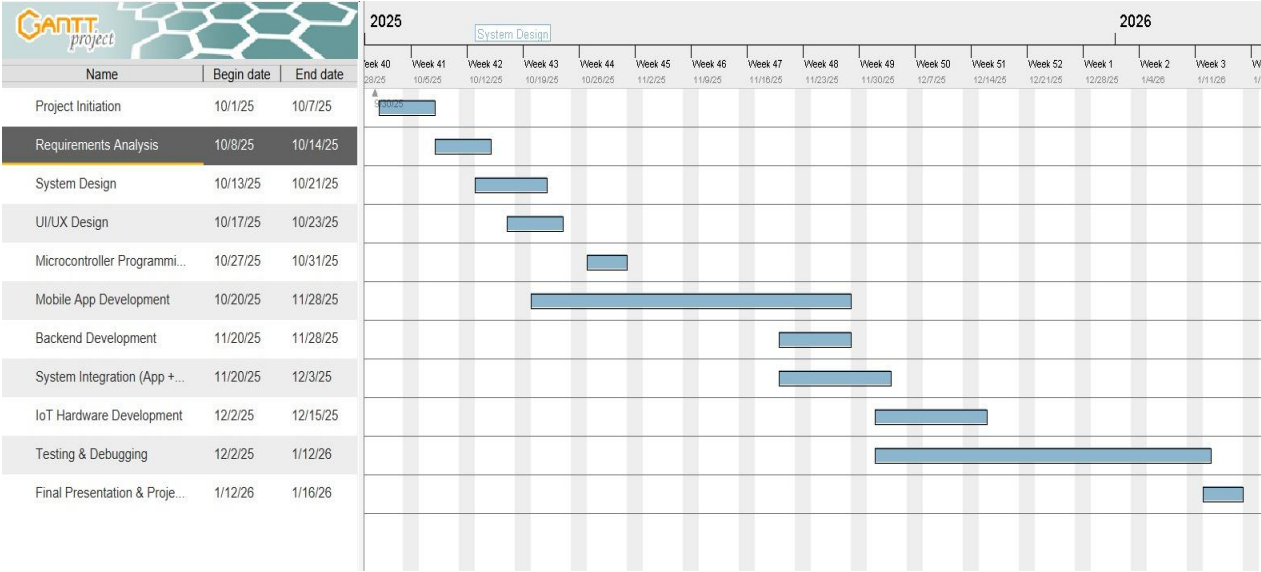


Figure 1-1 Gantt chart

CHAPTER 2

LITERATURE REVIEW

2.1 Definitions

We review the development and current landscape of collision-avoidance technologies within the automotive industry. Starting with the fundamental concepts of hazard detection and risk assessment, the discussion then moves to major active safety features such as Adaptive Cruise Control, forward-collision warning, and lane-departure prevention. This is followed by an examination of the sensor technologies—radar, visual systems, and infrared—and the decision-making algorithms that support real-time responses. Together, these topics provide the foundation for understanding how modern object-detection frameworks are shaping the future of vehicle safety systems.

Collision Avoidance

Collision avoidance plays a critical role in numerous transportation and control systems, such as air traffic management, automotive safety technologies, and robotic navigation. These systems work to prevent collisions between moving entities by either providing alerts to a human operator or triggering an automatic response, commonly known as an intervention. The strategies for decision-making and the criteria for evaluating collision risk can differ widely depending on the particular application. Although collision avoidance is relevant across multiple fields, this section concentrates specifically on its use within the automotive industry.

Automotive Collision Avoidance

Traffic accidents rank among the leading causes of injuries and fatalities worldwide. Consequently, automotive manufacturers have increasingly focused on developing driver assistance technologies to help mitigate these risks. One such technology is Adaptive Cruise Control (ACC), which automatically adjusts a vehicle's speed to maintain a safe following distance from the car ahead. ACC systems typically provide controlled deceleration, usually around -3 m/s^2 , and can alert the driver if additional action is necessary to prevent a collision. Drivers can generally activate or deactivate

these systems, which often disengage at lower speeds—typically below 40 km/h—as part of safety and comfort considerations.

In addition to ACC, modern vehicles are equipped with systems that either issue collision warnings or automatically apply the brakes when a potential hazard is detected. A key challenge for these systems is the accurate and reliable interpretation of complex, real-world driving environments. The presence of numerous dynamic and static objects demands advanced detection and classification capabilities, making automotive collision avoidance both technically sophisticated and highly complex.

Automotive Safety

Despite substantial advancements in vehicle crashworthiness—with modern cars reducing the likelihood of fatal injuries by up to 88% compared to vehicles from the 1990s—traffic accidents continue to cause numerous injuries and deaths. To further mitigate these incidents, there has been increased focus on the development and deployment of active safety systems. For instance, yaw control systems have been shown to reduce the risk of fatal injuries by up to 49%, depending on road conditions.

Collision mitigation (CM) systems play a pivotal role by using sensors to continuously monitor the environment ahead of the host vehicle. They provide essential information that supports decision-making processes, enabling countermeasures designed to prevent or lessen the severity of frontal collisions.

Also referred to as collision avoidance systems (CAS), these systems are equipped with environmental perception capabilities and can take proactive measures to prevent imminent collisions. Such measures may include alerting the driver, applying the brakes, or adjusting the vehicle's trajectory by exerting torque on the steering wheel. Examples of these systems include:

- **Lane Departure Systems:** These systems track lane markings to determine the vehicle's position within the lane. If the vehicle unintentionally drifts out of its lane, the system either issues a warning or actively applies steering corrections to bring it back on course.

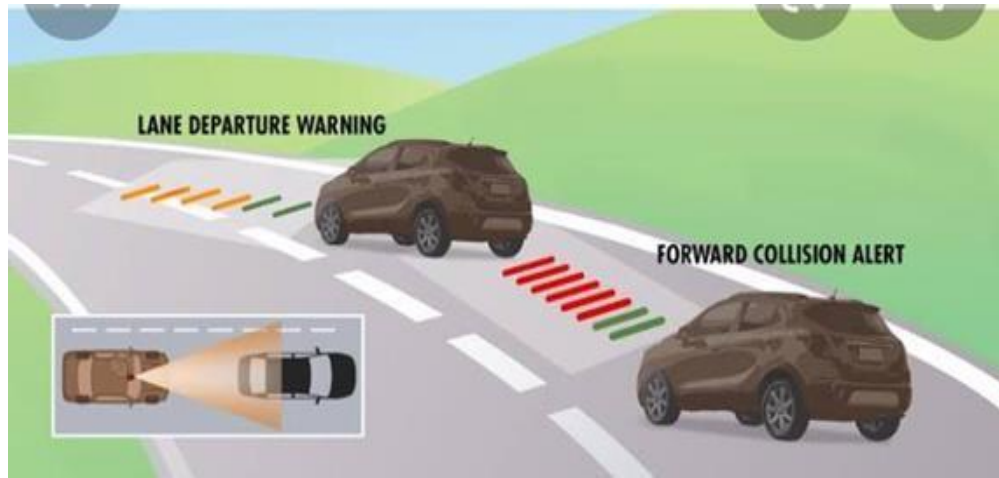


Figure 2-1 Lane departure system and forward collision system operations [3]

- **Forward Collision Mitigation Systems:** These systems continuously monitor the area directly in front of the vehicle and take action to prevent frontal collisions with obstacles.
- **Lane Change Aid Systems:** These monitor the vehicle's blind spots and the surrounding areas behind it, providing warnings or applying steering torque to facilitate safe lane changes and help avoid collisions.
- **Adaptive Cruise Control (ACC):** As mentioned earlier, ACC automatically adjusts the vehicle's speed according to traffic conditions to maintain a safe following distance.

In addition to these in-vehicle systems, the dedicated mobile application developed in this project enhances overall safety by enabling collaborative alerts. The application receives real-time detection data from the cloud, such as the presence of animals on the road, and notifies drivers in nearby vehicles. This extends hazard awareness beyond the vehicle's immediate sensors, providing drivers with additional time to react and helping prevent collisions.

In summary, collision mitigation systems depend on a range of sensors to constantly observe the vehicle's environment, enabling timely interventions to reduce the risk of collisions. The types of sensors employed vary depending on the system's specific functional requirements, and they include:

Vision System for Obstacle Recognition



Figure 2-2 Vision camera monitoring [4]

Vision Systems use one or more cameras paired with a microprocessor to analyze images in real time. These cameras function similarly to the human eye by capturing light within the visible spectrum. The performance of vision systems depends on factors such as sensor size, pixel resolution, lens quality, and dynamic range. Since they operate in three-dimensional environments, the associated image processing can be complex and computationally intensive. Typically, vision sensors update at around 25 frames per second.

There are two primary types of vision systems:

- **Single Camera Systems:** These use either monochrome (black-and-white) or color cameras and are commonly employed for lane-marking detection in lane-keeping assist functions.
- **Stereo Camera Systems:** These consist of two or more cameras working together to generate a three-dimensional view by combining images. Through triangulation, stereo systems can accurately determine distances to objects.

Advantages of Vision Systems:

- They provide high-resolution imaging, often using cameras with 640×480 pixels or higher. This enables accurate measurement of obstacle size and shape, as well as advanced object classification based on detailed image data.

Disadvantages of Vision Systems:

- Their effectiveness can decrease under poor lighting conditions, such as heavy rain, fog, or glare from wet roads.
- Image processing demands significant computational power, placing a heavy load on onboard processors.
- Vision systems are sensitive to dirt or obstructions on the camera lens, which can distort images and impair performance.

Infrared (IR) Vision Sensors



Figure 2-3 Night vision system operation in a car using an infrared sensor [5]

Infrared (IR) vision sensors, also known as thermal cameras, function similarly to conventional cameras but are specifically sensitive to infrared wavelengths associated with heat radiation. Unlike standard vision systems that capture visible light, IR sensors detect thermal emissions, allowing for improved perception in low-light or nighttime conditions. As a result, IR cameras are increasingly integrated into night vision systems to enhance driver visibility and safety.

These sensors are particularly effective at detecting warm-bodied obstacles, such as humans and animals, because they respond to the infrared wavelengths emitted by typical body temperatures. This capability also enables IR vision systems to perform accurate angular measurements and object classification based on thermal signatures. Despite these advantages, the relatively high cost of IR sensors compared to conventional visible-light cameras limits their widespread adoption in automotive collision mitigation systems.

2.2 Object Detection

Object detection is a computer vision technology focused on identifying and locating instances of specific object classes—such as humans, vehicles, and buildings—within digital images or video. It has been extensively developed in areas including pedestrian detection, scene text detection, face detection, edge detection, salient edge detection, and multi-category detection. Due to its versatility and effectiveness, object detection has seen widespread application across various domains, including military surveillance, transportation systems, medical imaging, and security. Practical implementations include security monitoring, traffic surveillance, autonomous driving, robotic vision, and drone-based scene analysis.

Object detectors are generally categorized into two main types: **two-stage detectors** and **one-stage detectors**. Two-stage detectors, such as Fast R-CNN, are renowned for their high accuracy in object recognition and precise localization. These systems operate in two steps: the first stage, called the Region Proposal Network (RPN), generates candidate bounding boxes for objects. In the second stage, features are extracted from each proposed bounding box using methods such as Region of Interest (RoI) pooling, followed by classification and bounding box regression. The basic architecture of two-stage detectors is illustrated in the figure below.

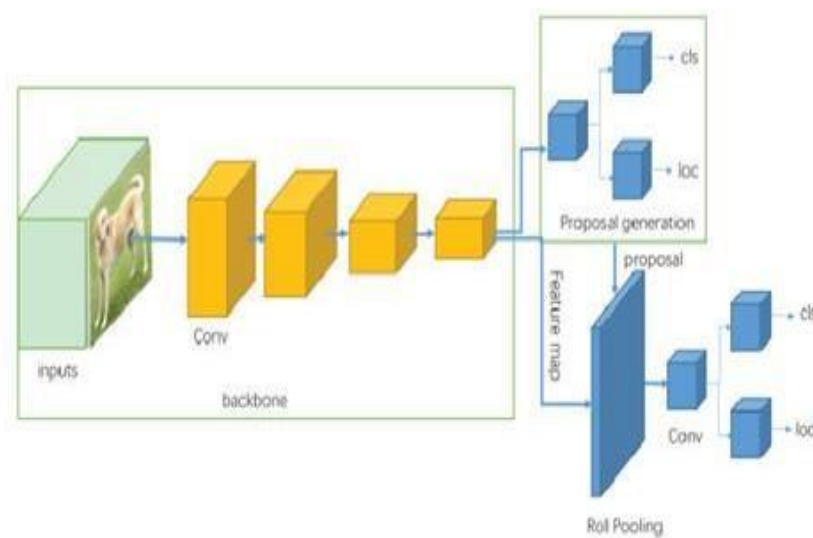


Figure 2-4 The basic architecture design of two-stage object detectors [6]

One-stage object detectors generate predicted bounding boxes directly from the input images, bypassing the regional proposal step. This approach makes these detectors highly time-efficient, making them suitable for real-time applications, as illustrated by the basic architecture in the figure below.

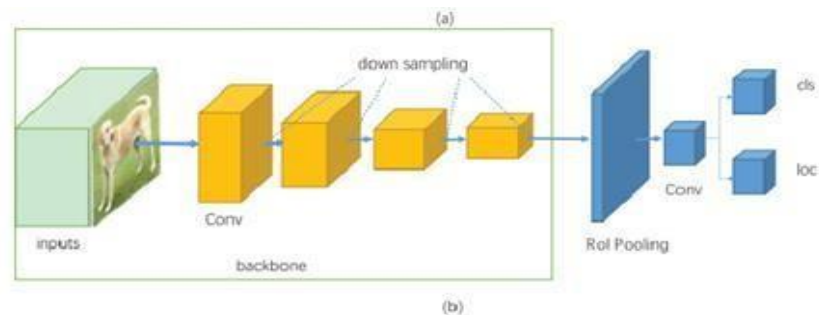


Figure 2-5 The basic architecture design of one-stage detectors [7]

Two-Stage Object Detectors

R-CNN

Region-based Convolutional Neural Network (R-CNN) decomposes object detection into four distinct stages. First, it uses a category-agnostic proposal generator (typically Selective Search) to extract candidate regions of interest (RoIs). Second, each proposed region is warped to a fixed size (227×227 pixels) and passed through a CNN to produce a 4,096-dimensional feature vector. Third, a set of one-vs-rest linear SVMs, each trained independently for a single object class, classifies these feature vectors. Fourth, a bounding-box regressor refines the coordinates of each proposal to improve localization accuracy.

During fine-tuning, proposals with Intersection over Union (IoU) ≥ 0.5 against any ground-truth box are considered positive, while those with IoU ≤ 0.3 are negative. Each ground-truth box “claims” the proposal with the highest IoU to ensure every object contributes at least one positive. This sampling strategy expands positive examples roughly $30\times$, reducing overfitting. R-CNN benefits from pre-training on large datasets (e.g., ImageNet) followed by domain-specific fine-tuning.[8]

Fast R-CNN

Fast R-CNN streamlines the R-CNN pipeline by combining multiple stages into a single end-to-end network trained using a multi-task loss that jointly optimizes classification and bounding-box regression. Instead of cropping and warping each RoI independently, the entire image is processed once through convolutional layers to generate a shared feature map. Each RoI is projected onto this map and reshaped to a fixed size via an RoI-Pooling layer, preserving spatial relationships without per-RoI warping. The pooled features feed into fully connected layers that output class scores and bounding-box offsets. Truncated singular value decomposition (SVD) can further accelerate inference by reducing computation in the fully connected layers.[8]

Faster R-CNN

Faster R-CNN advances Fast R-CNN by integrating a Region Proposal Network (RPN) into the shared convolutional backbone, removing the need for an external proposal generator. The RPN, a lightweight fully convolutional network, slides a 3×3 window over the last shared feature map. At each location, it predicts objectness scores and bounding-box deltas for k anchor boxes with varying scales (e.g., three sizes) and aspect ratios (e.g., three ratios). Sharing convolutional features between the RPN and downstream detector makes proposal generation faster and more accurate. Each anchor's predicted box is regressed toward its nearest ground-truth box, producing high-quality region proposals at multiple scales in a single forward pass .[8]

Mask R-CNN

Mask R-CNN extends Faster R-CNN to perform pixel-level instance segmentation alongside detection. It uses a ResNet-FPN (Feature Pyramid Network) backbone that constructs a multi-scale feature hierarchy via:

- **Bottom-up pathway:** A standard ResNet producing progressively coarser feature maps (stride 2 at each stage).
- **Top-down pathway:** Upsamples high-level, semantically rich maps and merges them with corresponding bottom-up maps using lateral 1×1 convolutions.

This creates a pyramid of feature maps at different scales, processed independently for classification, bounding-box regression, and mask prediction. A separate mask branch, implemented as a small FCN on each RoI, predicts binary segmentation at a fixed resolution (e.g., 28×28), enabling precise object outlines without compromising detection speed.[9]

One-Stage Object Detectors

YOLO (You Only Look Once)

YOLO reframes object detection as a single regression problem, mapping raw image pixels directly to bounding-box coordinates and class probabilities. The network processes the entire image in one forward pass, achieving real-time detection (~ 45 FPS on a Titan X GPU without batching). The input image is divided into an $S \times S$ grid. Each grid cell predicts B bounding boxes with a confidence score:

$$\text{Pr}(\text{object}) \times \text{IOU}(\text{predicted}, \text{ground truth})$$

and C conditional class probabilities. The feature extractor consists of 24 convolutional layers followed by two fully connected layers. Doubling the input resolution during pre-training (e.g., from 224×224 to 448×448) improves localization accuracy. YOLO tends to exhibit larger localization errors compared to two-stage detectors like Fast R-CNN and Faster R-CNN.[10]

YOLO v2

YOLO v2 enhances YOLO's speed and accuracy through architectural and training improvements:

- **High-Resolution Classifier:** Fine-tuning at 448×448 on ImageNet improves mean average precision (mAP) by $\sim 4\%$.
- **Convolutional Anchors:** Predicting objectness and class scores from convolutional feature maps for k anchor boxes improves recall by $\sim 7\%$, with a 0.3% drop in mAP.
- **Batch Normalization:** Accelerates convergence and regularizes training ($\sim 2\%$ mAP gain).
- **Dimension-Clustered Anchors:** Using k-means clustering for anchor initialization adds $\sim 5\%$ mAP.
- **Multi-Scale Training:** Random input resizing enforces scale invariance, maintaining ~ 40 FPS at high resolution.
- **Darknet-19 Backbone:** 19 convolutional layers with five max-pooling stages reduce computation while preserving accuracy.
- **Fine-Grained Feature Fusion:** Combining higher- and lower-resolution feature maps adds $\sim 1\%$ mAP.[11]

YOLO v3

YOLO v3 introduces:

- **Multi-Label Classification:** Independent logistic classifiers replace softmax to handle overlapping classes.
- **Feature Pyramid Predictions:** Detection heads operate on three scales for improved small-object recall.
- **Darknet-53 Backbone:** A deeper, residual-inspired 53-layer convolutional network enriches feature representation.
- **3D Tensor Outputs:** Each scale outputs a $(B \times (4 + 1 + C))$ -dimensional tensor encoding box offsets, objectness, and class probabilities.

- These improvements boost performance on varied object sizes, particularly small objects, while medium/large object mAP may plateau.[12]

YOLO v4

YOLO v4 optimizes the balance between speed and accuracy through:

- **Backbone (CSPDarknet53):** Cross-Stage Partial connections reduce computation and improve gradient flow.
- **Neck (SPP + PANet):** Spatial Pyramid Pooling captures context at multiple scales; Path Aggregation Network strengthens feature propagation.
- **Head (YOLO v3 style):** Retains the efficient detection head of YOLO v3.
- **Advanced Augmentations:** Mosaic (combines four images), Self-Adversarial Training (SAT), and Cross-Iteration BatchNorm (CiB).
- **Multi-Scale Inference:** Dynamically adjusts input size for scale-robust detection.
- YOLO v4 runs at ~65 FPS on a single GPU while achieving state-of-the-art accuracy for real-time systems.[13]

YOLO v5

YOLO v5, implemented in PyTorch by Ultralytics, introduces several engineering and performance optimizations:

- **Focus Module:** Slices and rearranges input tensors to reduce spatial resolution and increase channel depth.
- **Backbone (CSPDarknet):** Efficient CSP architecture balancing width, depth, and bottleneck layers.
- **Neck (PANet):** Enhances multi-scale feature fusion.
- **Head:** Three detection heads predict bounding boxes, objectness, and class probabilities at multiple scales.

- **AutoAnchor:** Learns optimal anchor dimensions via clustering during training.
- **Advanced Augmentations:** Mosaic & MixUp, Adaptive Image Scaling.
- **Loss Functions:** CIoU loss for bounding-box regression; label smoothing to prevent over-confident predictions.
- **Hyperparameter Evolution:** Optimizes training settings via genetic algorithms.
- YOLO v5s achieves over 100 FPS on modern GPUs, while larger variants (m, l, x) trade speed for higher mAP, matching or exceeding YOLO v4 accuracy.[14]

2.3 Collaborative Communication

Cloud Services for Data Analysis and Local Dissemination

Modern IoT systems leverage cloud platforms to ingest high-volume telemetry, perform real-time analytics, and disseminate insights to nearby devices. The leading platforms—Firebase, Google Cloud, and AWS—offer secure message ingestion, in-cloud processing, and edge runtime support.

The dedicated mobile application developed in this project serves as the user interface for the collaborative communication system. It receives real-time alerts from the cloud regarding detected obstacles, such as animals on the road, and notifies drivers in nearby vehicles. By continuously updating vehicle GPS coordinates, the application ensures that alerts are accurately targeted to vehicles within the relevant hazard zone, enhancing safety and reaction times while minimizing unnecessary distractions.

Firestore

- **Device Gateway:** Supports bidirectional HTTPS/FCM, HTTPS/FCM over WebSockets, and HTTPS endpoints with mutual TLS authentication and per-device IAM policies.
- **Rules Engine:** No SQL-like message filters route telemetry to targets such as Cloud Functions, Kinesis, DynamoDB, or S3 for storage, transformation, or ML inference.

- **Firebase IoT Greengrass:** Edge runtime deploys Cloud Functions functions and ML models to gateway devices, synchronizes shadows locally, and publishes processed events to the cloud or LAN devices.[15]

Google Cloud Firebase

- **Protocol Bridge:** Secure HTTPS/FCM and HTTPS ingestion with per-device JWT authentication, terminating at managed device registries.
- **Cloud Pub/Sub Integration:** Telemetry ingested into Pub/Sub topics ensures at-least-once delivery for real-time analytics via Dataflow, Cloud Functions, or custom services.
- **Cloud IoT Edge:** Extends Pub/Sub subscriptions and Dataflow pipelines to containerized edge devices, optionally leveraging Edge TPU accelerators for low-latency ML inference and local dissemination.[17]

Uplink: Delivering Detected Data from Device to Cloud

Once an object-detection node generates metadata or events, reliable transmission to the cloud is required. Common connectivity options include **LoRaWAN** and **cellular LTE**, each offering trade-offs in power consumption, range, throughput, and hardware complexity. This project focuses on the **SIM7600 family (Categories 1–4)**, particularly the Cat 4 variant.

The dedicated mobile application acts as the interface that receives the detection data after it is transmitted to the cloud. Once the vehicle’s detection node sends information about detected obstacles, such as animals on the road, the cloud processes and relays these alerts to the mobile application. The application then notifies drivers in real time, using both visual and audio cues, ensuring rapid awareness and allowing timely reactions to potential hazards. Continuous GPS updates from the vehicle ensure that alerts are accurately targeted to relevant nearby vehicles, enhancing both safety and system efficiency.

Here’s a fully reworded version of your text, keeping all the technical details intact while improving readability and flow:

LoRaWAN



Figure 2-6 AIS01-LB LoRaWAN[18]

LoRaWAN is a low-power, wide-area network (LPWAN) protocol operating in the unlicensed ISM frequency bands (433 MHz, 868 MHz, and 915 MHz). It is specifically designed to support battery-powered devices that transmit small data packets over long distances while minimizing energy consumption.

Data Rates & Spreading Factors: Depending on the chosen spreading factor (SF7–SF12), LoRaWAN end-nodes can transmit data at rates ranging from roughly 50 kbps (SF7) down to 0.3 kbps (SF12). Higher spreading factors improve link budget and sensitivity, thereby increasing communication range, but reduce throughput and lengthen time-on-air.

Coverage & Range: In dense urban areas, a single LoRaWAN gateway typically provides reliable coverage of 2–5 km. In suburban or rural line-of-sight scenarios, effective range can reach 10–15 km. This long-range capability makes LoRaWAN highly suitable for widely distributed sensor networks.

Node Hardware: Typical LoRaWAN sensor nodes combine a Semtech SX1276/78 RF transceiver with an ultra-low-power microcontroller (e.g., STM32L0 series). At maximum transmit power (+14 dBm), the SX1276 draws about 120 mA; during reception, it consumes around 9.9 mA; and in deep-sleep mode, current falls below 1 μ A. Antennas may be PCB-trace or external omnidirectional SMA units (~2.15 dBi gain). Regional duty-cycle regulations allow payloads up to 242 bytes, although practical messages are often under 50 bytes to preserve battery life.

Gateway & Network Server: Gateways use multi-channel concentrators (e.g., Semtech SX1301) that simultaneously monitor eight uplink channels and two downlink

channels. Received packets are forwarded via Ethernet or cellular backhaul to a LoRaWAN Network Server, where frames are decrypted using separate network and application session keys, de-duplicated, and reassembled if necessary. The resulting application payload, often in JSON format, is then sent to cloud endpoints over HTTPS/FCM or HTTPS for further processing.

Downlink Classes: LoRaWAN defines three device classes to balance power consumption and downlink latency:

- **Class A:** Each uplink transmission triggers two short receive windows, offering the lowest standby power consumption.
- **Class B:** Devices open scheduled receive windows synchronized with network beacons, which increases idle current slightly.
- **Class C:** Devices continuously listen for downlink messages except when transmitting, minimizing latency at the cost of higher power consumption.

SIM7600 Series (LTE Categories 1–4)

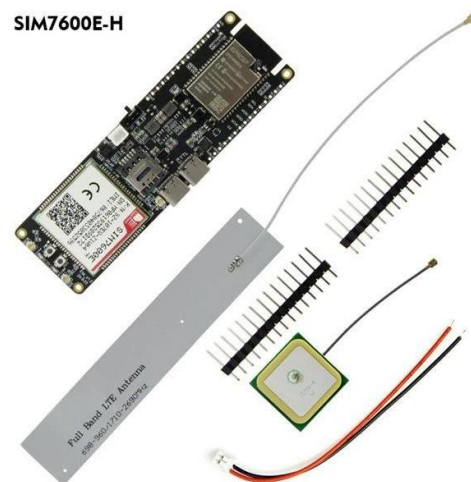


Figure 2-7 SIM7600E-H module ESP32-WROVER-B chip[19]

The SIM7600 series includes LTE modules compliant with 3GPP Release 9, spanning Categories 1 through 4. Each category defines maximum throughput, interface complexity, and power requirements [20]:

- **Category 1:** Supports up to 10 Mbps downlink and 5 Mbps uplink, using Frequency-Division Duplex (FDD) with a single antenna and half-duplex operation. This configuration minimizes hardware complexity and power draw, making Cat 1 ideal for basic IoT applications like telemetry or status reporting.
- **Category 2:** Offers 50 Mbps downlink and 25 Mbps uplink while maintaining a single-antenna design. It supports moderate-data applications such as periodic image uploads or firmware updates without significantly increasing power or complexity.
- **Category 3:** Delivers 100 Mbps downlink and 50 Mbps uplink, with options for single-antenna or 2×2 MIMO configurations. Cat 3 is suitable for higher-throughput needs, including video snippets or aggregated sensor streams, while keeping a reasonable balance between performance, power, and cost.
- **Category 4:** Provides up to 150 Mbps downlink and 50 Mbps uplink via 2×2 MIMO over a 20 MHz channel. While Cat 4 modules have more complex RF front-ends and higher peak current, they are ideal for applications requiring rapid transfer of large datasets, such as high-resolution images or object-detection metadata.

Hardware Characteristics (SIM7600 Cat 4):

- **Form Factor & Power:** LGA package (~42×38×4.2 mm); supply voltage 3.3–4.2 V; transmit current up to 2 A; receive ~300 mA; sleep current <10 µA via AT+CFUN commands.
- **RF Bands:** Supports multiple LTE FDD (e.g., B1, B3, B5, B7, B8, B20, B28) and TDD (e.g., B38, B40, B41) bands, enabling global or regional coverage.
- **Interfaces:** USB 2.0 (high-speed), UART (3.3 V TTL), PCM audio, GPIOs, I²C/SPI, SIM socket, and dual SMA antenna ports (main and diversity).
- **Embedded Stacks:** Includes TCP/IP, UDP, and HTTPS/FCM clients; TLS 1.2 support via QSSLCFG and QSSLOPEN AT commands.

Category Comparisons & Design Implications:

- **Throughput vs. Power:** Cat 4's high throughput (150/50 Mbps) requires more transmission power and complex RF front-ends (2×2 MIMO), whereas Cat 1's modest rates (10/5 Mbps) operate with a single antenna and simpler PCB layout.
- **Coverage & Sensitivity:** Lower categories often achieve slightly better coverage in challenging RF environments due to optimized link budgets and narrower bandwidths.
- **Cost & Complexity:** Cat 1 modules are lower cost and simpler, ideal for battery-powered IoT devices; Cat 4 modules are more expensive but essential when large image or video datasets must be transmitted quickly.

2.4 The ADS Mobile Application: Architecture and Role in Collaborative Safety

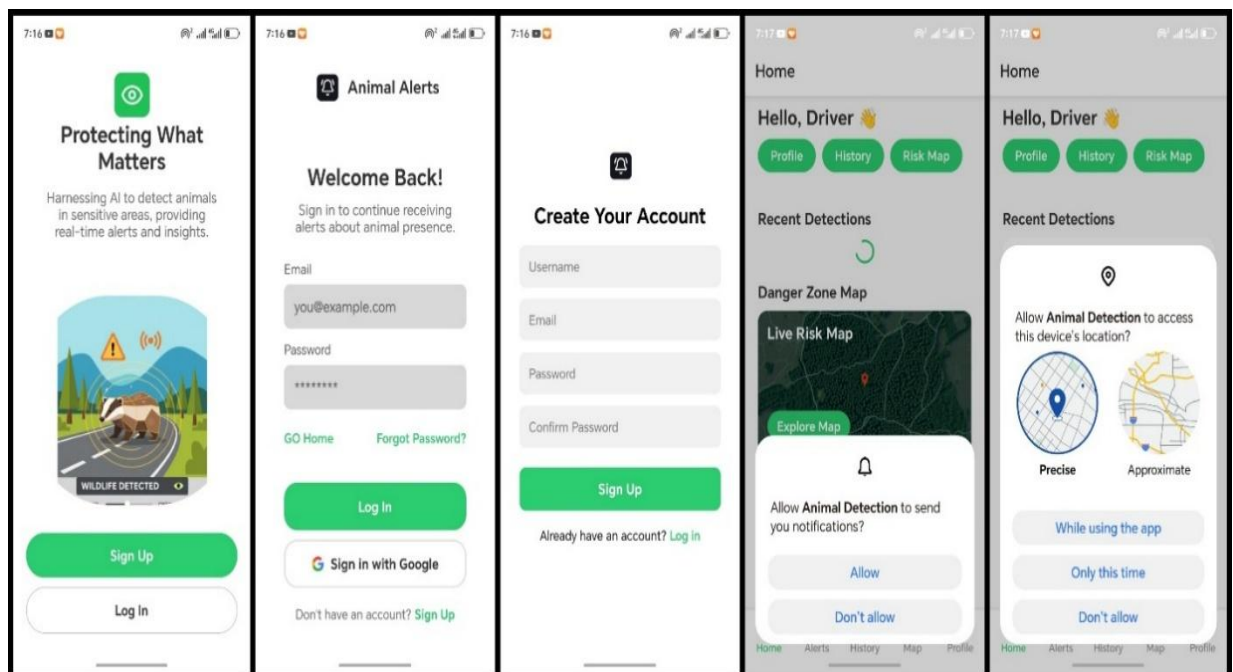


Figure 2-8 Application project

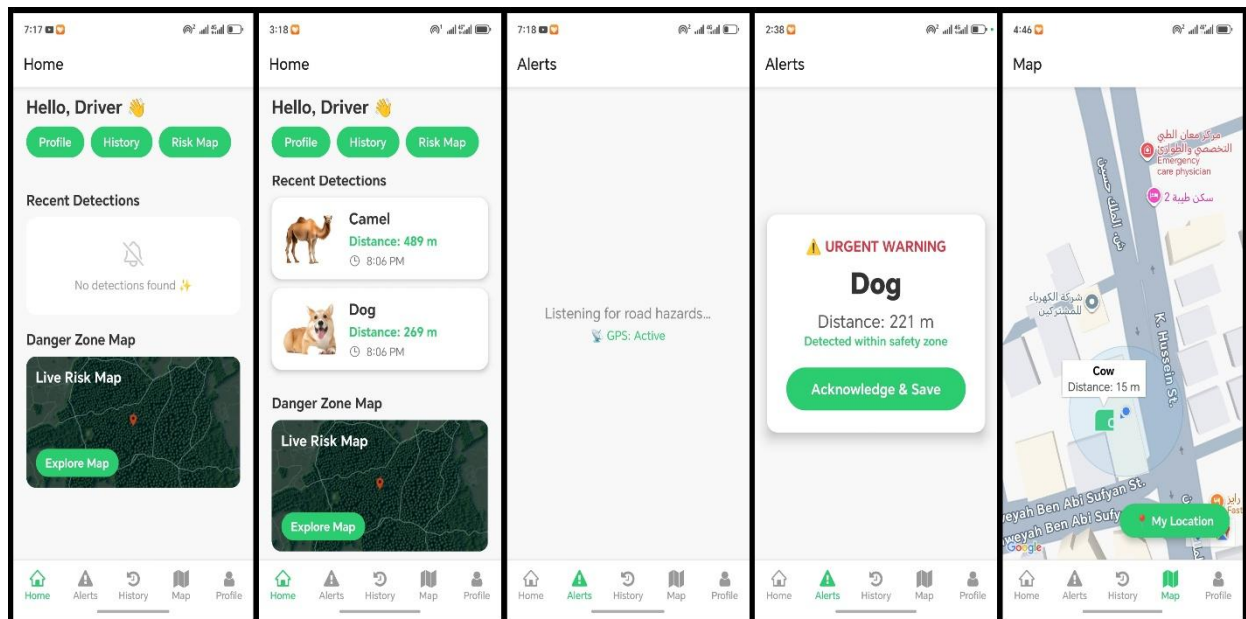


Figure 2-9 From a project

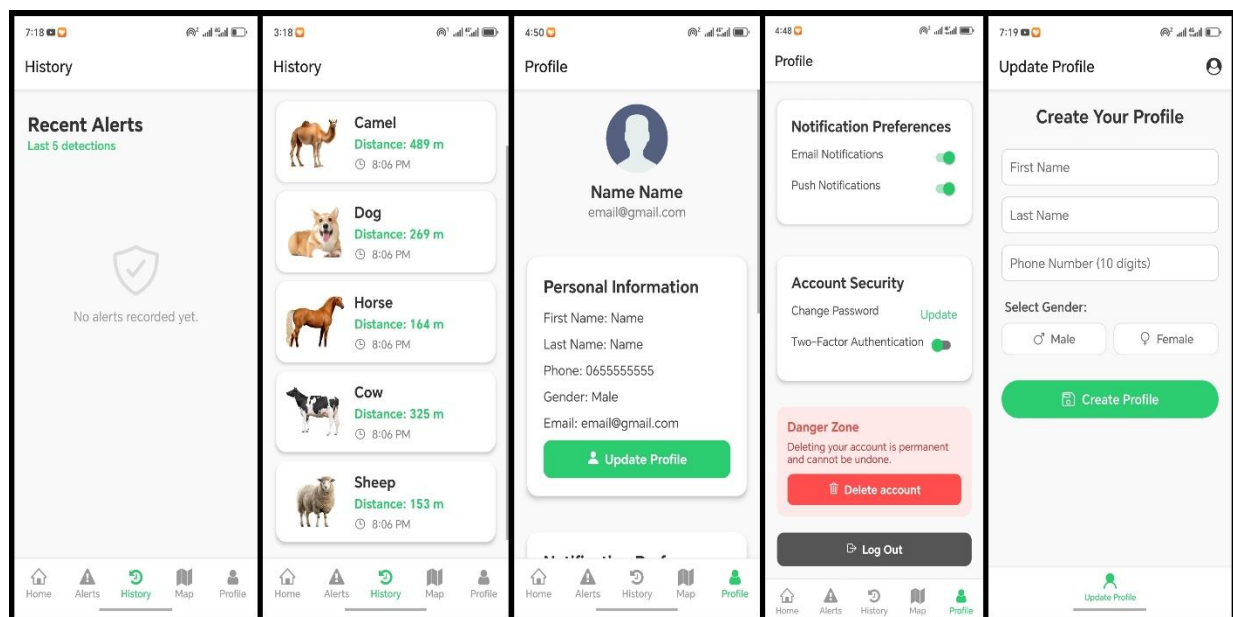


Figure 2-10 From a project

Conceptual Framework and System Integration

The ADS mobile application serves as the critical human–machine interface in the collaborative safety ecosystem. It transforms raw detection data from the edge system into actionable safety information for drivers. The application operates on a real-time publish–subscribe model, leveraging Firebase services to enable instantaneous alert dissemination and coordinated vehicle awareness.

Technology Stack and Implementation

The application was developed using Expo React Native, selected for its fast development workflow and cross-platform capabilities, and integrated with a full Firebase backend architecture:

Authentication & Security Layer

- Firebase Authentication using email/password and social login options
- Secure user session management with token-based authentication
- Optional two-factor authentication for enhanced security

Real-Time Data Infrastructure

- Firebase Realtime Database for instantaneous alert delivery and vehicle location synchronization
- Optimized data structure for high-frequency reads and writes in safety-critical operations
- Persistent WebSocket connections for continuous real-time updates

Cloud Messaging & Notification System

- Firebase Cloud Messaging (FCM) for high-priority push notifications
- Background notification handling to ensure alerts are received even when the app is closed
- Priority-based routing to guarantee immediate delivery of wildlife hazard alerts

Cross-Platform Development Strategy

Using the Expo framework allowed:

- Unified deployment of a single codebase to both Android and iOS
- Faster prototyping and development iterations
- Native-level performance while retaining development efficiency
- Simplified app building and distribution through Expo services

User Experience Design and Core Features

The application follows a safety-first design structure consisting of four main modules:

Authentication & Onboarding

- Smooth login and registration workflows using Firebase Authentication
- Support for google login providers
- Persistent and secure credential management across sessions

Real-Time Alert Interface

- Alerts triggered instantly through Firebase Cloud Messaging
- Live alert state synced with Firebase Realtime Database
- High-visibility screen layouts optimized for driver attention and readability

Historical Data Visualization

- Retrieval of past alerts from Firebase Realtime Database
- When a new alert appears in the Real-Time view, its data should be saved as a document in Firestore.
- Cached data for improved performance
- Organized historical logs with timestamps and location metadata

User Management & Preferences

- Real-time synchronized user profiles via Firebase
- Persistent notification and app preferences
- Secure account configuration through Firebase Auth APIs

System Performance and Reliability

The combined Firebase–Expo architecture provides:

- Low-latency alert delivery through FCM priority channels
- High service availability supported by Google’s cloud infrastructure
- Automatic backend scaling during peak user load
- Consistent experience across platforms with reliable performance

The ADS mobile application demonstrates how modern mobile technologies—Expo React Native and Firebase—can be integrated to deliver a scalable, robust, and safety-critical system. It effectively bridges edge-level wildlife detection with real-time driver awareness to enhance collaborative road safety.

2.5 Recent Work

Recent advances in object detection have substantially improved autonomous system capabilities, particularly for road safety and real-time vehicle perception. These developments are evident in both research and real-world implementations that enhance detection reliability under diverse conditions.

A notable contribution is the **Fast-COS framework**, which introduces the Reparameterized Attention Vision Transformer (RAViT). This model incorporates Reparameterized Multi-Scale Depth-Wise Convolution (RepMSDW) and Reparameterized Self-Attention (RepSA) to boost object detection performance for autonomous driving. Evaluations on the BDD100K and TJU-DHD datasets show that Fast-COS achieves an AP50 of 57.2% and GPU inference speeds up to 75.9% faster than models like FCOS and YOLOF, demonstrating strong potential for real-time deployment [21].

Another approach merges YOLO and Faster R-CNN into a hybrid model, yielding a 5–7% improvement in accuracy over standard YOLO models. Trained on over 10,000 traffic images, this hybrid model balances detection speed and precision, particularly in congested urban environments [22].

Efforts have also focused on robustness under adverse conditions. Ulyanov et al. (2023) proposed a YOLOv8-based detection model fine-tuned with combined datasets, including ACDC and DAWN, improving object detection in snow, rain, and fog. This highlights the importance of environmental diversity in training [23].

Furthermore, a team from Incheon National University developed an **IoT-enabled 3D object detection system** that fuses LiDAR point clouds with RGB images using a YOLOv3-based model. The system achieved 96% 2D and 97% 3D detection accuracy on the Lyft dataset, demonstrating strong potential for integration into autonomous driving platforms [24].

While recent work has largely focused on enhancing the perception capabilities of individual vehicles, our project, the Animal Detection System (ADS), introduces a collaborative dimension to road safety. This work moves beyond a single-vehicle context by integrating a real-time animal detection module with a cloud-based alert dissemination network. A key component of this system is a dedicated mobile application that delivers these collaborative warnings to drivers. This creates a safety ecosystem where a hazard detected by one vehicle can proactively warn multiple drivers in the vicinity, addressing a critical gap in current research that often overlooks inter-vehicle communication and practical driver-alert interfaces.

CHAPTER 3

METHODOLOGY

In this chapter, a comprehensive description is provided of the implementation of our project, which involves the development of a real-time animal detection and alert system aimed at mitigating animal–vehicle collisions. The system is designed to be applicable to all types of vehicles and is capable of detecting specific animal species that are commonly encountered on roads in Jordan, namely sheep, cows, dogs, and horses. Following detection, the system issues prompt alerts to both the vehicle driver and other nearby vehicles within a radius of 1 km Through an application we created. The implementation process was structured in accordance with the main objective and the corresponding specific objectives presented below:

3.1 Construct Framework

- To design and implement a system capable of real-time detection of specific animal species and of issuing alerts to drivers in order to prevent potential collisions.

3.2 Develop System Architecture

- To design and develop an object detection model capable of recognizing dog, horse, cow, camel and sheep in real-time video streams.
- To deploy the developed detection system on a Raspberry Pi 5 platform and assess its real-time performance.
- To implement a communication framework utilizing the SIM7600 module to transmit detection alerts to a cloud server, which subsequently disseminates notifications to nearby vehicles within a 1 km radius.

To accomplish these objectives, a YOLOv5-based object detection model was trained using a custom dataset comprising images of the target animal species. The complete system was implemented on a Raspberry Pi 5 and employed a Raspberry Pi NoIR camera to capture real-time video streams. Upon detection of an animal, the system

triggers an onboard speaker to alert the driver and simultaneously uses the SIM7600 communication module to transmit detection information to a cloud server. The server then relays alert notifications to vehicles located within a 1 km radius. Using only a computer, without working on hardware.

The system we designed was comprised of basically two stages namely;

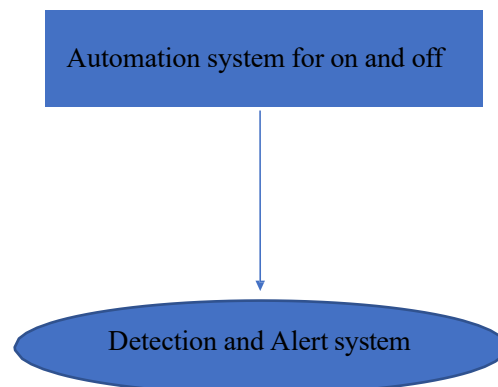


Figure 3-1 representation of our 2-stage system

3.3 Analyze and Design the System

Animal-vehicle collisions (AVCs) pose a significant threat to both human and animal safety, particularly on rural and forest-adjacent roadways. According to the World Health Organization (WHO), road traffic accidents are expected to become the fifth leading cause of death globally by 2030 [2]. While most focus is placed on vehicle-to-vehicle or vehicle-to-pedestrian accidents, animal-related collisions account for a notable portion of total incidents. In Jordan, for example, the 2020 Traffic Police report estimated that approximately 6% of all road traffic accidents involved animals. Although this percentage may seem small, these collisions often result in severe injuries, fatalities, vehicle damage, and wildlife mortality. Traditional warning systems such as static signs or manually triggered alert systems are ineffective in addressing the dynamic nature of animal crossings. They neither adapt to the presence of animals in real time nor provide precise location-based alerts to approaching vehicles. This project proposes a smart, low-cost, and real-time animal detection and alert system that leverages computer vision, edge computing, and cloud-based networking to address this safety challenge. The solution will consist of a camera-based detection system powered by a Raspberry Pi 5, capable of identifying animals using a lightweight object detection

model (YOLOv5), alerting nearby drivers through onboard audio warnings, and transmitting incident data to the cloud for collaborative communication with other nearby vehicles. The goal is to create a scalable and deployable platform that can operate effectively in varied environmental conditions and help reduce animal-vehicle collisions while contributing to data-driven wildlife protection efforts.

3.4 Observe and Evaluate the System

At the outset of the project, a general animal detection strategy was adopted through the use of a YOLOv5 model pre-trained on the COCO dataset. The COCO dataset was selected because of its large-scale nature, high-quality annotations, and its capacity to recognize more than 80 object categories, including commonly encountered animals such as dog, horse, cow, camel and sheep, which correspond to the target classes of this project. Furthermore, the COCO pre-trained model demonstrates a high accuracy of approximately 95%, rendering it a robust baseline for object detection applications [26].

This initial configuration enabled the rapid development of a functional prototype without the requirement to collect or annotate substantial amounts of data from the beginning. Nevertheless, since the COCO model was trained to detect a wide variety of object classes—many of which fall outside the scope of this project—it became necessary to exclude irrelevant classes and constrain the detection process exclusively to the animals of interest, namely dog, horse, cow, camel and sheep [27].

As the project advanced, multiple approaches were investigated to tailor both the detection model and the dataset processing pipeline, ensuring that the system detected only the relevant animal classes.

Filtering at the Inference Stage (Post-Detection)

The first approach consisted of executing the complete COCO pre-trained model without modification and subsequently filtering the inference results using a Python-based post-processing script. Under this approach, detections for all object classes were initially generated; however, only those corresponding to the target animal categories were retained as valid detections, while all remaining classes were

disregarded and did not trigger any alerts. Although this approach was straightforward to implement, it still required the execution of the full model across all object classes, which negatively affected processing time and overall system efficiency

```
# Define target classes
target_classes = ['sheep', 'cow', 'horse', 'dog']
target_class_ids = [model.names.index(name) for name in target_classes]

# Set confidence threshold
CONFIDENCE_THRESHOLD = 0.75

# Initialize video capture (0 = default webcam; change if using Pi camera or
# USB cam)
cap = cv2.VideoCapture(0)

if not cap.isOpened():
    print("Error: Cannot open camera")
    exit()

while True:
    ret, frame = cap.read()
    if not ret:
        print("Error: Cannot read frame")
        break

    # Run YOLOv5 inference
    results = model(frame)

    # Convert to pandas dataframe
    detections = results.pandas().xyxy[0]

    # Filter by class and confidence
    filtered = detections[
        (detections['name'].isin(target_classes)) &
        (detections['confidence'] >= CONFIDENCE_THRESHOLD)
    ]

    # Draw results
    for _, row in filtered.iterrows():
        xmin, ymin, xmax, ymax = int(row['xmin']), int(row['ymin']), int(row
        ['xmax']), int(row['ymax'])
        label = f"{row['name']} {row['confidence']:.2f}"

        cv2.rectangle(frame, (xmin, ymin), (xmax, ymax), (0, 255, 0), 2)
        cv2.putText(frame, label, (xmin, ymin - 10),
            cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)

    # Show the frame
    cv2.imshow('YOLOv5 Real-Time Detection', frame)

    # Press 'q' to quit
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
```

Figure 3-2 snippet of post-processing Class Filtering code

Filtering Before Detection at the Inference Stage

The second approach sought to eliminate irrelevant classes prior to the detection stage during inference. The objective was to restrict the model's detection scope at inference time to include only the target classes, thereby reducing redundant computational overhead. However, because the model architecture was pre-trained to recognize all object classes, implementing such class-level filtering necessitated custom modifications or configuration adjustments during inference. Although this approach was intended to enhance computational efficiency, it presented practical limitations arising from the inherent design of the pre-trained model [28].

Custom Class Fine-tuning

The final and most effective approach consisted of constructing a custom dataset that included exclusively the five target animal classes. All irrelevant classes and images were eliminated, resulting in a substantial reduction in dataset size and a simplification of the training process. Using the Roboflow online platform [29], the refined dataset was systematically organized, annotated, and exported, which significantly streamlined the data preparation stage. The detection model was subsequently retrained using this curated dataset and was specifically optimized to recognize only dog, horse, cow, camel and sheep. This approach led to improvements in both detection speed and accuracy. In addition, the reduced dataset size decreased the time required for alert generation, thereby enhancing overall system efficiency and suitability for real-time deployment. A representative screenshot of the dataset configuration in Roboflow is provided below to illustrate this process.

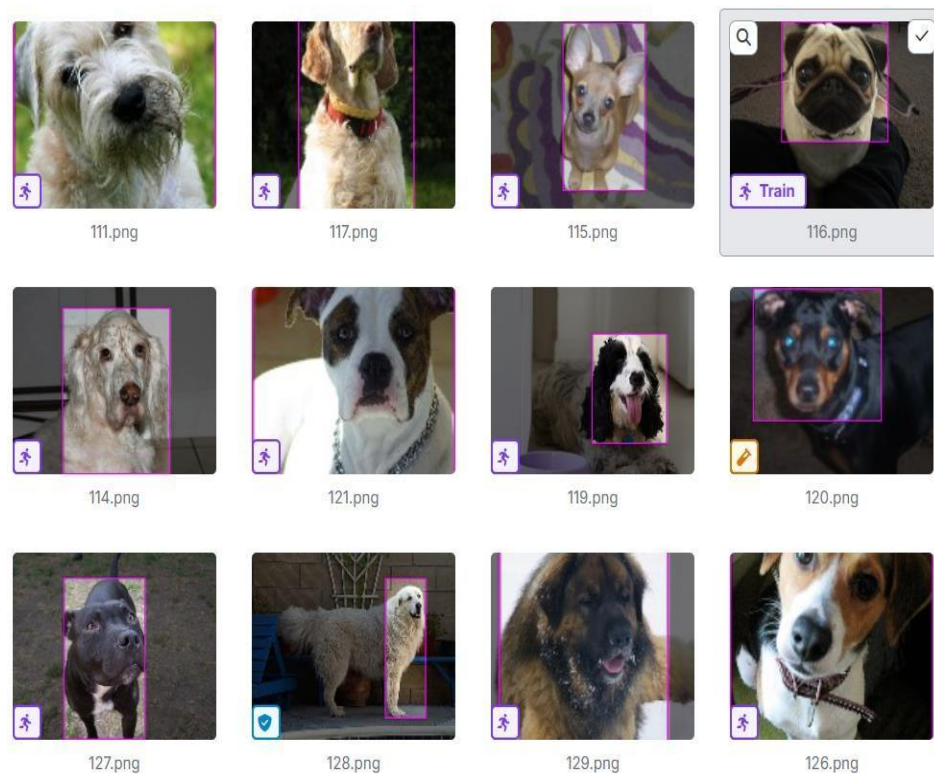


Figure 3-3 dataset setup on Roboflow

To support this, the following table summarizes the number of images used from the COCO dataset after filtering and sorting:

Table 3-1 Number of images used in the final custom dataset

Image type	Overall collected images	Images after sorting
Dog	5,000	4,250
Horse	2,226	1,892
Camel	4000	3000
Cow	2,447	2,079
Sheep	2,220	1,887
Total	15,893	13,108

By narrowing down the dataset to only the relevant animal classes, we significantly reduced the number of training images compared to the full COCO dataset. While the original COCO dataset contains approximately 118,000 training images, our final filtered dataset included only 13,108 images. This represents a reduction of about 107,892 images, or nearly 91.43% of the original dataset size. Such a substantial reduction helped improve model training efficiency and inference speed without compromising the accuracy for our specific detection targets.

Model architecture specification

YOLOv5 was selected due to its high inference speed compared to other object detection models, such as Efficient Det, which makes it particularly suitable for real-time applications and enables the processing of a large number of frames per second. This characteristic makes YOLOv5 well suited to the requirements of the proposed project, which necessitates the analysis of a high volume of frames within a limited time interval, as illustrated in the figure below.

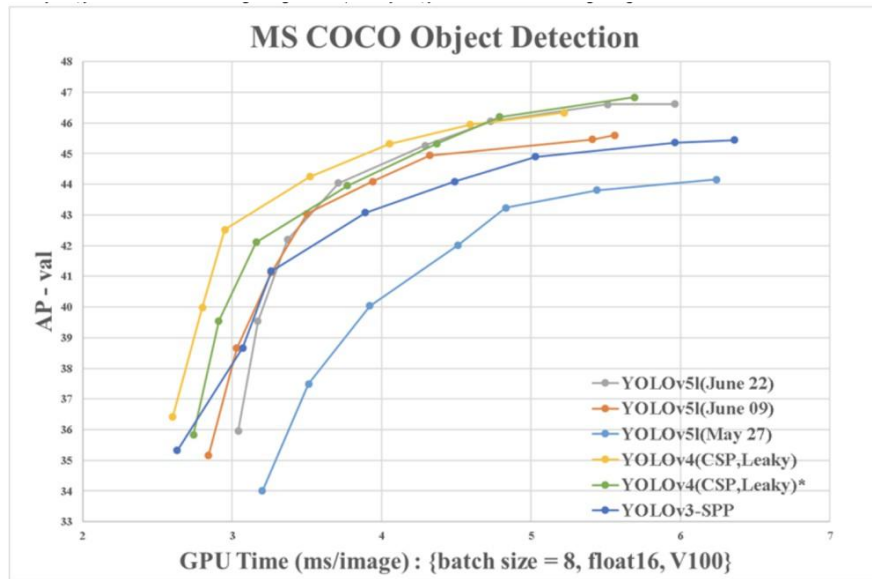


Figure 3-4 A graph showing YOLOv5 superior performance on AP and frames per second [30]

NB: Average precision (AP) and frames per second are high compared to other object detectors as clearly observed from the graph that there was a 18% increase in Frames per second and 16% increase in average precision compared to YOLOv5.

Google Colab

Google Colab (commonly referred to as Google Colab) is a free, cloud-based Jupyter Notebook environment provided by Google that enables users to write and execute Python code directly through a web browser. It provides access to substantial computing resources, including up to 12 GB of RAM (expandable to 25 GB in certain cases) and approximately 358 GB of disk storage. This platform was particularly valuable in overcoming the hardware limitations of our personal computers, which were restricted to 8 GB of RAM. Consequently, Google Colab offered a practical and cost-effective solution for executing both the training and inference stages of our object detection model [31].

Model parameters

Our model was developed using a set of carefully chosen parameters to ensure optimal performance for our specific use case. These parameters were selected based on experimentation and resource availability, and they contributed significantly to the accuracy and efficiency of the model. The final configuration is summarized in the table below.

YOLOv5 Training Parameter	Value
Batch Size	16
Image Size (Width × Height)	640 × 640
Number of Epochs	150
Learning Rate	0.01
Loss Function	YOLOv5 built-in
Dataset Split (Train / Validation)	80% / 20%
Number of Classes	5
Number of Channels	3 (RGB)

Table 3-2 : Table showing the parameters used for model training to obtain an optimal performance

NB: These are the optimal parameters that were achieved by analyzing the required accuracy and precision.

Model Training

Using our generated and filtered custom dataset containing five target animal classes (dog, horse, cow, camel and sheep), we trained a YOLOv5 model within the Google Colab environment. The steps followed during training are summarized as follows:

- Installed all required dependencies and cloned the YOLOv5 repository.
- Uploaded and prepared the custom dataset in YOLO format, with annotations in .txt files.

- Configured a custom YAML file specifying class names, dataset paths, and number of classes.
- Executed the training script using the YOLOv5 training pipeline.

```
1 path: C:\Users\Lenovo\Desktop\animal_dataset_clean
2 train: images/train
3 val: images/val
4
5 names:
6   0: dog
7   1: cow
8   2: sheep
9   3: horse
10  4: camel
11
```

Figure 3-5 : snippet of the YAML file

After running the initial training session, we observed a noticeable gap between the performance metrics we expected and the results obtained, particularly in terms of confidence scores and mean Average Precision (mAP). To improve the model's performance, we applied the following modifications:

- Increased the input image resolution from 512×512 to 640×640 to improve training efficiency.
- Lowered the batch size to optimize GPU memory usage and enable more stable training on the Google Colab environment.
- Reduced the number of epochs to prevent overfitting and reduce total training time, while still allowing sufficient learning cycles.

During training, we closely monitored key evaluation metrics such as mean Average Precision (mAP) and **loss**, which served as indicators of the model's accuracy and convergence over time.

Mean Average Precision (@mAP)

Mean Average Precision (mAP) is a widely accepted evaluation metric used in the field of computer vision to assess the performance and robustness of object detection models. It is derived from four fundamental parameters: the confusion matrix, intersection over union (IoU), precision, and recall. These parameters are used to compute the average precision (AP) for each individual class.

The mAP is then calculated as the mean of the AP values across all classes, providing a single performance indicator for multi-class object detectors.

The value of mAP ranges between 0 and 1, where:

- $\text{mAP} = 1$ indicates perfect detection (optimal performance),
- $\text{mAP} = 0$ indicates complete failure (poor performance).

A higher mAP value reflects a better ability of the model to accurately detect and classify objects across all target classes, making it a critical benchmark for evaluating model effectiveness.

NB: When we trained our model, from our first training we obtained $\text{mAP}@0.5$ of 0.54 and with model architecture adjustments as earlier discussed we obtained $\text{mAP}@0.5$ of 0.83.

3.5 Model Performance Evaluation

These results demonstrate a clear improvement in the overall performance of the proposed model.

Loss

The loss function in object detection models, including YOLOv5, plays a crucial role in guiding the training process by measuring

the discrepancy between the predicted outputs and the actual ground truth. It consists of three main components:

- Localization loss, which calculates the error between the predicted bounding box coordinates and the actual object positions.
- Confidence loss, which evaluates the objectness score to determine the presence of an object within a predicted bounding box.
- Classification loss, which measures how accurately the detected object is classified into the correct category.

A low loss value indicates more accurate predictions and a more efficient training process. Generally, loss values below 0.2 are considered acceptable for object detection tasks and reflect effective model learning.

NB: For our first training, we obtained very large loss values for our object detectors but after considerable adjustments earlier discussed, the loss values reduced to values below 0.2 which evidenced faster and more accurate model training of our object detectors.

Comparison with Other Models

To evaluate the performance of our chosen object detection model, we also trained and tested alternative object detectors, including YOLOv10 (pre-trained on COCO) and Detectron2. These models were selected based on their popularity and performance in recent object detection research.

The objective of this comparison was to assess how our YOLOv5 model performed in relation to these counterparts in terms of key evaluation metrics, particularly mean Average Precision (mAP)

and loss. While each model has its unique architecture and optimization strategy, YOLOv5 demonstrated superior balance between speed and accuracy, making it well-suited for real-time animal detection in our application.

In contrast, models such as YOLOv10 and Detectron2, although effective in terms of accuracy, required significantly more computational resources and training time. Their relatively higher loss values and slower inference speed made them less practical for deployment on resource-constrained environments like the Raspberry Pi used in our project.

This comparison further reinforced the choice of YOLOv5 as the most suitable model for achieving real-time, efficient, and accurate animal detection within the scope of our system.

3.6 SIM7600 Operation

The SIM7600 is a 3G/4G LTE modem that supports GPS functionality. It is used in this project to retrieve GPS data (latitude and longitude) and send it over cellular networks to the cloud for processing and alerting. The key tasks for SIM7600 are:

- Retrieving GPS data using AT commands.
- Providing cellular connectivity for data transmission to the cloud (in this case, Firebase).

Hardware Setup

- SIM7600 Module is connected to the Raspberry Pi via the USB port (e.g., `/dev/ttyUSB3`).
- SIM card with cellular data access is inserted into the SIM7600 module for internet connectivity.
- The Raspberry Pi also has a camera to capture images for

object detection (such as animals) using YOLOv5.

Installing Required Libraries

- To communicate with the SIM7600 module, we need to install the **pyserial** library to send AT commands and read the responses over a serial connection. Install the pyserial library:

```
pip install pyserial
```

Figure 3-6

- To access the Internet, we will use the Wvdial, and before running it, we will need to modify the connection settings [32]:

```
GNU nano 4.8 /etc/wvdial.conf Modified
[Dialer Defaults]
Modem = /dev/ttyUSB2
Baud = 115200
Phone = *99#
Username = "net"
Password = "net"
Stupid Mode = 1
Dial Command = ATDT
APN = net.orange.jo
```

Figure 3-7 Snippet of Wvdial configuration file

Sending AT Commands to SIM7600

The SIM7600 module communicates via AT commands. To get the GPS coordinates, we send the command `AT+CGPSINFO`. [33]

Here's the process to retrieve GPS data:

1. Connect the SIM7600 to the Raspberry Pi via USB.
2. Send the AT command `AT+CGPSINFO` to retrieve the

GPS coordinates (latitude and longitude).

3. Parse the response to extract the GPS data.

3.7 Publishing on and Receiving from Firebase

Retrieve GPS Data from SIM7600

Below is the Python code to retrieve the GPS data from the SIM7600 module and publish it on Firebase:

```
1 from awscrt import io, mqtt
2 from awsiot import mqtt_connection_builder
3 import time, json, serial
4
5 # ===== AWS IoT Core Settings =====
6 ENDPOINT = "a3q0wl5uajvgim-ats.iot.eu-north-1.amazonaws.com" # AWS IoT endpoint
7 CLIENT_ID = "Raspberry-Pi" # Unique client/device identifier
8 TOPIC = "raspberrypi/data" # MQTT topic to publish data
9 CERT_PATH = "/5db0e1f01de27a29fc57acd6cb5e5c0027ab9a2c036ec128d7f181fb0e67942-certificate.pem.crt" # Device certificate path
10 KEY_PATH = "/5db0e1f01de27a29fc57acd6cb5e5c0027ab9a2c036ec128d7f181fb0e67942-private.pem.key" # Device private key path
11 ROOT_CA = "/AmazonRootCA1.pem" # Root CA certificate path
12
13 # ===== GPS Settings =====
14 GPS_PORT = "/dev/ttyUSB3" # Serial port for SIM7600 GPS
15 GPS_BAUD = 115200 # Baud rate for GPS communication
16 SERIAL_TIMEOUT = 0.5 # Serial timeout in seconds (less than 1s)
17
18 # 1) Open serial port once for communication with SIM7600
19 ser = serial.Serial(GPS_PORT, GPS_BAUD, timeout=SERIAL_TIMEOUT)
20
21 # 2) Turn on GPS once
22 ser.write(b'AT+CGPS=1,1\r\n') # Enable GPS with hot start
23 time.sleep(3) # Initial wait time for GPS fix acquisition (cold start)
24
25 def get_gps():
26     """Request GPS coordinates quickly without toggling GPS power each time."""
27     ser.write(b'AT+CGPSINFO\r\n') # Request GPS information
28     time.sleep(0.3) # Reduced wait time from 1s to 0.3s for faster response
29     resp = ser.read(ser.in_waiting or 1).decode(errors='ignore')
30     for line in resp.splitlines():
31         if line.startswith('+CGPSINFO:'):
32             parts = line[len('+CGPSINFO:').:].strip().split(',')
33             if len(parts) >= 4 and parts[0] and parts[2]:
34                 # Convert GPS coordinates from ddmm.mmmm format to decimal degrees
35                 raw_lat = float(parts[0])
36                 lat = int(raw_lat // 100) + (raw_lat % 100) / 60
37                 if parts[1] == 'S': lat = -lat # Southern hemisphere correction
38                 raw_lon = float(parts[2])
39                 lon = int(raw_lon // 100) + (raw_lon % 100) / 60
40                 if parts[3] == 'W': lon = -lon # Western hemisphere correction
41                 return {
42                     "latitude": round(lat, 6),
43                     "longitude": round(lon, 6),
44                     "fix_status": "valid"
45                 }
46     # Return default invalid GPS coordinates if no valid data is found
47     return {
48         "latitude": 0.0,
49         "longitude": 0.0,
50         "fix_status": "invalid"
51     }
52
```

Figure 3-8

```

53 # ===== MQTT Connection Setup =====
54 event_loop_group = io.EventLoopGroup(1)
55 host_resolver = io.DefaultHostResolver(event_loop_group)
56 client_bootstrap = io.ClientBootstrap(event_loop_group, host_resolver)
57
58 # Build a secure MQTT connection using certificates
59 mqtt_connection = mqtt_connection_builder.mtls_from_path(
60     endpoint=ENDPOINT,
61     cert_filepath=CERT_PATH,
62     pri_key_filepath=KEY_PATH,
63     client_bootstrap=client_bootstrap,
64     ca_filepath=ROOT_CA,
65     client_id=CLIENT_ID,
66     clean_session=False,
67     keep_alive_secs=30
68 )
69
70 def main():
71     print("Connecting to AWS IoT Core...")
72     mqtt_connection.connect().result() # Establish MQTT connection
73     print("Connected!")
74
75     try:
76         while True:
77             gps = get_gps() # Fast GPS data retrieval
78             # Prepare the message payload
79             message = {
80                 "device": CLIENT_ID,
81                 "message": "WARNING: Real-time animal detection",
82                 "location": gps,
83                 "timestamp": time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime()),
84             }
85             print("Publishing:", message)
86             # Publish message to the MQTT topic with QoS 1 (at least once)
87             mqtt_connection.publish(TOPIC, json.dumps(message), qos=mqtt.QoS.AT_LEAST_ONCE)
88             time.sleep(5) # Delay between publishes; adjust as needed
89     except KeyboardInterrupt:
90         print("\nShutting down...")
91         mqtt_connection.disconnect().result() # Disconnect cleanly
92         # 3) Turn off GPS when shutting down
93         ser.write(b'AT+CGPS=0\r\n')
94         ser.close() # Close serial connection
95         print("Done.")
96
97 if __name__ == "__main__":
98     main()

```

Figure 3-9 Snippet Python code to retrieve GPS data and publish it on Firebase

3.8 Explanation of the Code:

1. GPS Data Retrieval:

- a. The `get_gps_data()` function sends the AT command `AT+CGPSINFO` to retrieve the GPS coordinates from SIM7600.
- b. The GPS data is parsed and returned as latitude and longitude.

2. Firebase Communication:

- a. The Raspberry Pi sends GPS and detection events to Firebase using HTTPS requests (REST API) or the Firebase Admin SDK. These events are stored in Firebase Realtime Database to enable real-time processing and synchronization.

- b. The `publish_data()` function prepares the GPS/detection payload (timestamp, coordinates, detected class, confidence) and writes it to a structured path in Firebase Realtime Database (e.g., `/events` and `/liveVehicles`).
- 3. HTTPS over TLS:

All communication between the Raspberry Pi, Firebase backend, and the mobile application is encrypted using TLS/SSL, ensuring secure data transmission.

Receiving messages from Firebase

Below is an example approach to receive alerts through Firebase on the mobile application using Firebase Cloud Messaging (FCM), and to optionally listen to Firebase Realtime Database for live event updates.

```
from awscrt import io, mqtt
from awsiot import mqtt_connection_builder
import json

ENDPOINT = "a3q0wl5uajvgim-ats.iot.eu-north-1.amazonaws.com"
CLIENT_ID = "Raspberry-Pi"
TOPIC = "raspberrypi/data"
CERT_PATH = "/5db0e1f01de27a29fc57acd6cb5e5c0027ab9a2c036ec128d7f181f1b0e67942-certificate.pem.crt"
KEY_PATH = "/5db0e1f01de27a29fc57acd6cb5e5c0027ab9a2c036ec128d7f181f1b0e67942-private.pem.key"
ROOT_CA = "/AmazonRootCA1.pem"

def on_message_received(topic, payload, **kwargs):
    print(f"Received message: {json.loads(payload)}") # Debug line

# Build MQTT connection
mqtt_connection = mqtt_connection_builder.mtls_from_path(
    endpoint=ENDPOINT,
    cert_filepath=CERT_PATH,
    pri_key_filepath=KEY_PATH,
    ca_filepath=ROOT_CA,
    client_id=CLIENT_ID,
    on_message_received=on_message_received
)

# Connect and subscribe
print("Connecting...")
connect_future = mqtt_connection.connect()
connect_future.result()
print(f"Subscribing to {TOPIC}...")
subscribe_future, _ = mqtt_connection.subscribe(
    topic=TOPIC,
    qos=mqtt.QoS.AT_LEAST_ONCE,
    callback=on_message_received
)
subscribe_future.result()
print("Waiting for messages...")

# Keep the connection alive
while True:
    pass
```

Figure 3-10 Subscription code snippet to receive messages from Firebase

3.9 Cloud Integration with Firebase

This logic receives FCM push notifications for urgent alerts and can also subscribe to live changes in Firebase Realtime Database to display real-time event information in the app.

Cloud Integration with Firebase

Once GPS and detection data are generated on the Raspberry Pi, they are transmitted securely to the Firebase backend over HTTPS via the SIM7600 cellular module. Active events and live vehicle-location updates are written to Firebase Realtime Database to support low-latency processing and real-time synchronization.

In parallel, confirmed detection records are archived in Firebase Firestore for persistent storage, historical analysis, and reporting. Each Firestore document can store the detected animal class, confidence score, timestamp, GPS coordinates, and (optionally) an image snapshot URL stored in Firebase Storage.

Firebase Cloud Functions are triggered on new detection events to perform proximity filtering (e.g., identifying vehicles within a 1 km radius using the Haversine formula). Relevant drivers are then notified through Firebase Cloud Messaging (FCM).

Once the GPS data is retrieved from SIM7600, it needs to be sent to Firebase for processing. Here are the steps involved:

Firestore Setup [34]

1. Create a Thing in Firestore:

- a. Register your Raspberry Pi device as a Thing in Firestore.
- b. Download the Firestore Authentication and Security Rules certificates (device certificate, private key, and root CA) to authenticate the device.

2. Topics:

- a. raspberry/data: This topic is used for sending GPS data from the Raspberry Pi to Firestore.

3. IoT Policy:

- a. Define an IoT policy that allows devices to publish, subscribe, connect, and receive data from these topics.

Example IoT policy:

Policy file details

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "*",
      "Resource": "*"
    }
  ]
}
```

Figure 3-11 Example Firebase Security Rules configuration

Firestore Realtime Database Triggers and Cloud Functions for Proximity Filtering

After the detection event is written to Firestore Realtime Database, a Firestore Cloud Function is automatically triggered. It calculates the distance between the detection point and nearby vehicles using their latest GPS coordinates stored in the database.

1. Proximity Filtering Using Firestore Cloud Functions:

- a. The Cloud Function applies a distance calculation (e.g., Haversine formula) to identify vehicles within a 1 km radius.
- b. Vehicles that fall inside the danger zone are selected as alert recipients.

```
1 SELECT * FROM 'raspberry/+'
```

Figure 3-12

2. Alert Dissemination:

- a. After selecting nearby vehicles, the Cloud Function sends high-priority push notifications via Firestore Cloud Messaging (FCM).
- b. The mobile application receives these alerts and presents a clear audible/visual warning to the driver.

```

1 import json
2 import math
3 import boto3
4
5 # Initialize AWS resources
6 dynamodb = boto3.resource('dynamodb')
7 iot_client = boto3.client('iot-data')
8
9 # DynamoDB table name
10 TABLE_NAME = "DevicesLocation"
11 # Topic template for alerts
12 ALERT_TOPIC_TEMPLATE = "raspberrypi/alert"
13
14 def haversine(lat1, lon1, lat2, lon2):
15     """
16     Calculate the great-circle distance between two points
17     on the Earth surface using the Haversine formula.
18     """
19     R = 6371 # Earth radius in kilometers
20     dlat = math.radians(lat2 - lat1)
21     dlon = math.radians(lon2 - lon1)
22     a = math.sin(dlat/2)**2 + math.cos(math.radians(lat1)) * math.cos(math.radians(lat2)) * math.sin(dlon/2)**2
23     c = 2 * math.atan2(math.sqrt(a), math.sqrt(1-a))
24     return R * c
25
26 def lambda_handler(event, context):
27     # If the event is from an IoT Rule, the payload is the event itself
28     payload = event
29
30     # Extract device info
31     device_id = payload['device_id']
32     lat = float(payload['latitude'])
33     lon = float(payload['longitude'])
34     msg_type = payload.get('type', 'event')
35
36     # Reference to DynamoDB table
37     table = dynamodb.Table(TABLE_NAME)
38
39     # Always update the device's location
40     table.put_item(Item={
41         'device_id': device_id,
42         'latitude': lat,
43         'longitude': lon
44     })
45
46     # If this is an event (detection), find nearby devices and alert them
47     if msg_type == "event":
48         response = table.scan()
49         devices = response['Items']
50
51         for device in devices:
52             other_id = device['device_id']
53             if other_id == device_id:
54                 continue # Skip the sender
55
56             other_lat = float(device['latitude'])
57             other_lon = float(device['longitude'])
58             distance = haversine(lat, lon, other_lat, other_lon)
59
60             if distance <= 1:
61                 topic = ALERT_TOPIC_TEMPLATE.format(other_id)
62                 message = {
63                     "from_device": device_id,
64                     "distance_km": distance
65                 }
66                 iot_client.publish(
67                     topic=topic,
68                     qos=1,
69                     payload=json.dumps(message)
70                 )
71
72     return {'statusCode': 200, 'body': 'Alerts sent '}
```

Figure 3-13 Snippet Cloud Functions code to filter sending alerts to devices are within 1km

3.10 Sequence Diagram of the Detection and Alert Process

illustrates the sequence of operations in the Animal Detection and Alert System, starting from capturing video frames and performing animal detection on the edge device, followed by transmitting detection data to the cloud server, and finally delivering alert notifications to the mobile application.

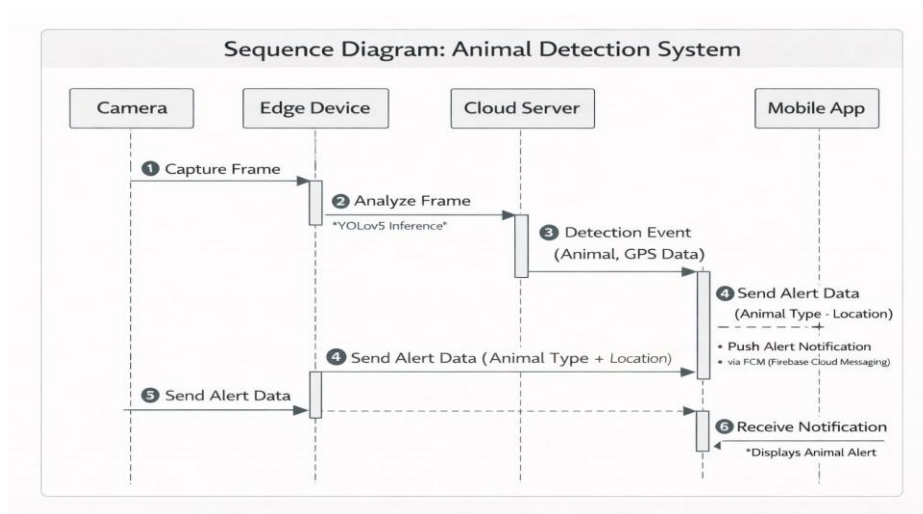


Figure 3-14 Sequence Diagram

3.11 Security Considerations

1. Secure Communication:

HTTPS over TLS/SSL ensures secure communication between the Raspberry Pi, Firebase services, and the mobile application.

2. Access Control:

Firebase Authentication manages user identity (app users), while Firebase Security Rules control read/write permissions for Realtime Database and Firestore. Service credentials are protected, and only authorized devices/app users can write or read relevant data.

CHAPTER 4

ANALYSIS AND DESIGN

4.1 System Overview and Use Case Diagram

The Use Case Diagram provides a high-level functional overview of the proposed Animal Detection and Alert System (ADS). It illustrates the main interactions between system actors and the core functionalities offered by the system, without exposing implementation or technical details. This diagram helps in clearly defining the system boundaries and understanding how different users and components interact with the system.

The primary actor of the system is the Driver, who benefits from real-time alerts when animals are detected on or near the roadway. The detection process is performed automatically by the Detection System, which continuously analyzes video streams captured by the camera using the YOLOv5 object detection model. Upon detecting an animal, the system generates a local audio alert and sends detection data to the cloud through the cellular communication module.

The Cloud Server acts as an intermediary entity responsible for receiving detection events, processing location-based filtering, and distributing alerts to nearby vehicles. The Mobile Application serves as the user interface that delivers visual and audible notifications to drivers, ensuring timely awareness of potential hazards.

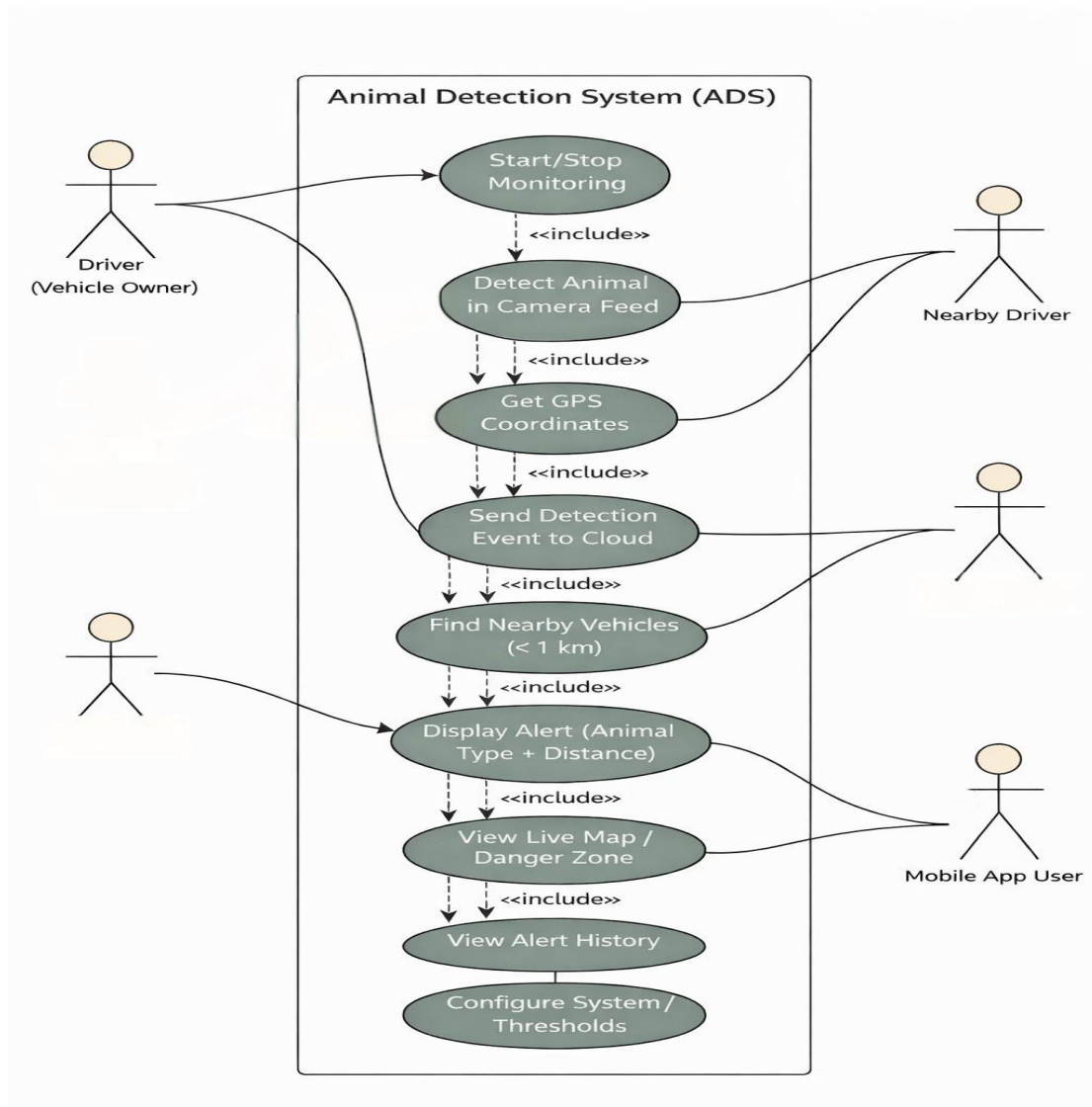


Figure 4-1 Use case

4.2 Data flow Diagram

The Data Flow Diagram (DFD) illustrates the flow of information within the proposed Animal Detection and Alert System. It presents how data is generated, processed, stored, and transmitted between system components, providing a clear representation of system functionality at the data level.

As shown in Figure 4.2, the process begins with a continuous video stream captured by the camera and processed by the detection module using the YOLOv5 model to identify animals in real time. Once an animal is detected, a detection event is generated and immediately triggers a local audio alert to notify the driver

coordinates is transmitted to the Firebase Cloud platform, where the event is stored in the Detection Events database. The cloud system then processes this information by comparing the detection location with vehicle location data stored in the Vehicle Locations database.

Based on proximity filtering, alerts are generated and distributed to nearby vehicles through cloud-based notification services. These alerts are delivered to nearby drivers, providing early warnings and enhancing road safety through collaborative communication.

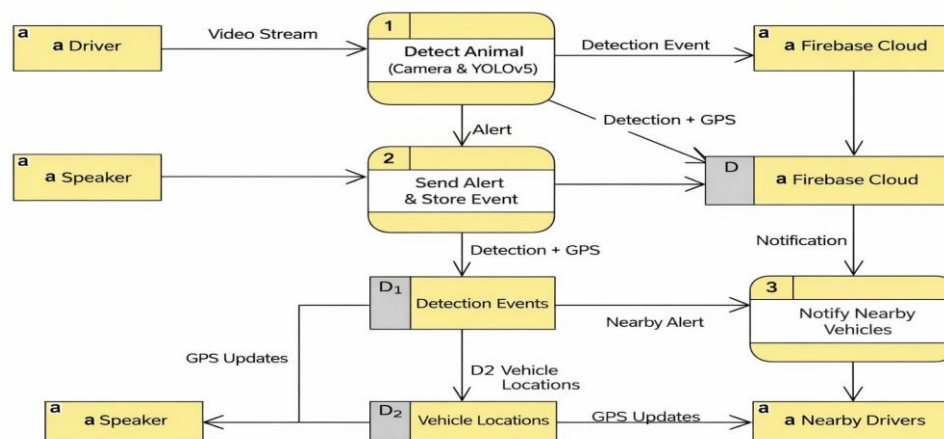


Figure 4-2 Data Flow Diagram of the Animal Detection and Alert System

4.3 Entity Relationship Diagram (ERD)

The Entity Relationship Diagram (ERD) represents the logical database structure of the proposed Animal Detection and Alert System. It defines the main entities, their attributes, and the relationships between them, providing a clear view of how data is organized and stored within the system.

As illustrated in Figure 4.3, the Vehicle entity represents registered vehicles participating in the system. Each vehicle is associated with a single Device, which corresponds to the embedded detection unit installed in the vehicle. The Device entity is responsible for generating detection events when animals are identified using the onboard camera and detection model.

Detection information is stored in the Detection_Event entity, which records details such as the detected animal type, confidence score, geographic location, and timestamp. Based on each detection event, one or more Alert records are generated to represent warning messages issued by the system.

The Mobile_User entity represents drivers using the mobile application. Each user is linked to a vehicle and may receive multiple notifications. The Notification entity tracks alert delivery details, including notification status and reception time, enabling reliable monitoring of alert dissemination.

This structured database design supports efficient event tracking, location-based alert filtering, and scalable data management for real-time and historical analysis.

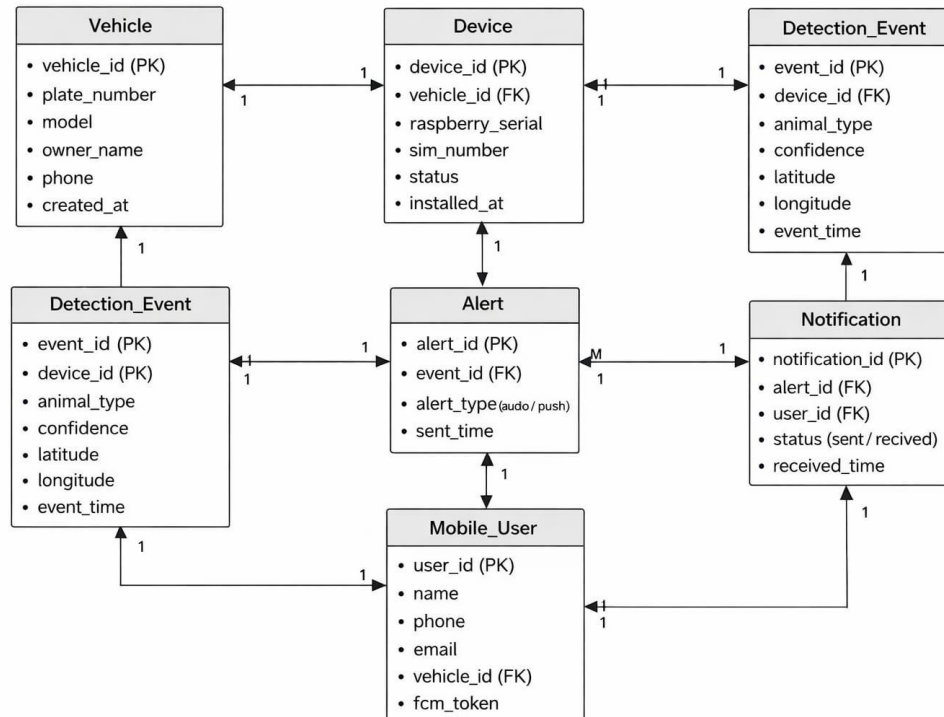


Figure 4-3 Entity Relationship Diagram (ERD) of the Animal Detection and Alert System

4.4 Design Requirements

Problem Statement

Animal–vehicle collisions (AVCs) represent a serious risk to both human life and wildlife, particularly on roads located near rural, agricultural, or forested regions. According to the World Health Organization (WHO), road traffic accidents are projected to become the fifth leading cause of death worldwide by the year 2030 [2]. Although the majority of safety research concentrates on vehicle-to-vehicle or vehicle-to-pedestrian crashes, collisions involving animals constitute a significant portion of overall incidents.

In Jordan, the Traffic Police Department reported in 2020 that approximately 6% of all recorded road accidents involved animals. While this percentage may appear relatively

small, animal-related collisions frequently result in severe consequences, including human injuries or fatalities, extensive vehicle damage, and loss of wildlife. These incidents also impose economic burdens and ecological harm.

Conventional warning methods, such as fixed roadside signs or manually activated alert systems, are insufficient for addressing the unpredictable and dynamic nature of animal crossings. Such solutions lack real-time adaptability and fail to provide precise, location-specific warnings to approaching vehicles.

To address this challenge, this project proposes a smart, low-cost, and real-time animal detection and alert system that integrates computer vision, edge computing, and cloud-based networking technologies. The proposed solution employs a camera-based detection mechanism powered by a Raspberry Pi 5, utilizing a lightweight object detection model (YOLOv5) to identify animals near roadways. Upon detection, the system provides immediate audio alerts to drivers and transmits incident data to a cloud server to enable collaborative warning dissemination to nearby vehicles.

The primary objective of this platform is to deliver a scalable and deployable system capable of operating under diverse environmental conditions, reducing animal–vehicle collisions, and contributing to data-driven wildlife protection and traffic safety initiatives.

To effectively implement the proposed system and address the outlined problem, the following design requirements must be satisfied:

1. Real-Time Detection

- The system shall continuously capture video frames using a Raspberry Pi Noir camera and process them with minimal latency.
- Object detection must be completed within 300 milliseconds per frame to ensure prompt warning delivery.

- Detection performance must remain reliable during both daytime and nighttime operation, adapting to varying lighting conditions such as dawn, dusk, and vehicle headlights.
- All video processing shall be performed locally using edge computing to eliminate reliance on internet connectivity or cloud-based inference.

2. High Detection Accuracy

- The object detection model must achieve a mean Average Precision (mAP) of 83%, as validated through prior training and evaluation on relevant datasets.
- The system shall accurately recognize animals commonly encountered near roadways, including dog, horse, cow, camel and sheep.
- A low false positive rate must be maintained to prevent unnecessary or distracting alerts, particularly in visually complex environments.

3. Embedded Deployment

- All computation and system execution shall be performed entirely on a Raspberry Pi 5 equipped with 8 GB of RAM, without the use of external servers or high-end GPUs.
- The detection model must be optimized in size and complexity (e.g., YOLOv5) to ensure a balance between detection accuracy and processing efficiency on embedded hardware.
- The camera shall be securely mounted and configured with a suitable field of view for forward-facing road monitoring.

4. Immediate Local Alerting

- Upon detecting an animal, the system shall immediately activate an audio alert using a connected speaker or buzzer.
- The alert volume must be sufficient to notify the driver inside the vehicle while avoiding excessive disturbance.

- Repeated alerts shall be generated if the detected animal remains within the defined detection zone.

5. Cloud-Based Data Transmission

- A SIM7600 4G cellular module shall be integrated to enable communication in regions without Wi-Fi connectivity.
- For each detection event, the system shall transmit a data packet containing the timestamp, GPS coordinates, detected animal class, and an image snapshot to a remote cloud server.
- All data transmissions must be secure, encrypted, and reliable to ensure data integrity and prevent loss.

6. Collaborative Networking

- Upon receiving a detection event, the cloud server shall invoke a Cloud Functions function to identify other deployed systems within a 1 km radius.
- Relevant warning information shall be forwarded to these systems to proactively alert drivers approaching the hazard area.
- This collaborative mechanism shall utilize GPS-based geofencing logic and support bi-directional communication for acknowledgment and system status updates.

7. Location Tracking and Logging

- The Raspberry Pi system shall periodically update its GPS location, even in the absence of detection events, to maintain continuous situational awareness.
- All detection events and location updates shall be logged locally and optionally synchronized with the cloud for redundancy and analytical purposes.

8. Data Storage and Visualization (Optional Extension)

- The cloud backend shall support structured storage of received detection data in a database.

- A visualization dashboard may be implemented to display incidents on a geographic map, analyze temporal trends, and evaluate road safety risks across different regions.

9. Environmental Robustness

- The system shall operate reliably under various environmental conditions, including dust, rain, fog, and extreme temperatures.
- The camera lens must be protected from debris such as dust or mud to preserve image quality.
- All wiring, connectors, and power components shall be enclosed securely to ensure durability during long-term outdoor deployment.

10. Cost Efficiency and Scalability

- The total cost per unit, including the Raspberry Pi, camera, SIM module, speaker, and power supply, shall remain below \$500 to enable large-scale deployment, particularly in developing regions.
- The system design shall allow for straightforward installation and maintenance, requiring minimal technical expertise.

4.5 Engineering Standards

To ensure compliance with modern technological practices, safety requirements, and interoperability principles, the proposed system adheres to established engineering standards throughout its design and implementation. These standards govern wireless communication, networking protocols, hardware integration, software development, and information security, thereby enhancing system reliability, scalability, and compatibility with future intelligent transportation systems.

1. Wireless Communication Standards

- **3GPP LTE (4G) – Release 13 and above**

The SIM7600 module conforms to LTE standards defined by the 3rd Generation Partnership Project (3GPP), ensuring compatibility with current mobile network infrastructures and reliable high-speed data transmission.

- **IEEE 802.11 b/g/n (Wi-Fi)**

Although cellular communication is the primary mode, optional Wi-Fi support is provided for system configuration and data upload. Compliance with IEEE 802.11 standards guarantees consistent and secure wireless communication.

2. Internet Protocols and Data Transmission

- **IPv4 / IPv6**

Support for both protocol versions ensures flexible and future-proof network connectivity.

- **HTTP/HTTPS (RFC 2616, RFC 2818)**

Communication between the edge device and the cloud backend utilizes HTTP and HTTPS, with HTTPS employed for encrypted data transfer.

- **HTTPS/FCM (ISO/IEC PRF 20922)**

HTTPS/FCM is considered for future system extensions involving lightweight, real-time machine-to-machine communication.

4. Software and Programming Standards

- **PEP 8 (Python Enhancement Proposal 8)**

Python code adheres to the PEP 8 style guide to ensure consistency, readability, and maintainability.

- **IEEE 829 – Software Test Documentation**

Testing documentation follows IEEE 829 guidelines to ensure systematic validation and verification of all system modules.

5. Camera and Imaging Standards

- **ITU-R BT.709**

The camera sensor complies with this standard for image capture and color representation, ensuring consistency with computer vision models.

- **MJPEG and H.264 Video Encoding**

These compression standards are used for efficient video capture and transmission during detection or debugging.

6. GPS and Geolocation Standards

- **NMEA 0183 Protocol**

GPS data transmission follows the NMEA 0183 standard, enabling reliable parsing and integration.

- **WGS84 Coordinate System**

All geographic data is referenced using the WGS84 global coordinate framework.

7. Cloud and Web API Standards

- **RESTful API Design (RFC 7231)**

Backend services are implemented using RESTful principles to support scalability and interoperability.

- **JSON (RFC 8259)**

All exchanged data is formatted using JSON for lightweight and platform-independent communication.

4.6 Realistic Constraints

The design and deployment of the proposed system are subject to several practical constraints that influence performance, cost, sustainability, and operational reliability.

1. Economic Constraints

- **Total System Cost**

Each unit must remain under a deployment budget of \$200 to ensure scalability. The current bill of materials includes:

- Raspberry Pi 5 (8 GB): \$150
- Raspberry Pi Noir Camera: \$45
- SIM7600 4G Module: \$130
- MicroSD Card (128 GB): \$10
- Speaker and Amplifier: \$15
- **Total Estimated Cost:** $\approx$ \$350

- **Data Usage and Cost**

Operating at 2.8 fps, with detections every 10 seconds and 0.8 KB per event:

- 8,640 detections/day
- 6.9 MB/day
- $\approx$ 207 MB/month

With an average data cost of \$2/GB, monthly connectivity cost remains below \$0.50 per unit.

- **Edge Processing Advantage**

On-device inference eliminates cloud compute costs, saving approximately \$10/month/unit compared to cloud-based image processing.

2. Environmental Constraints

- **Power Consumption**

- Raspberry Pi 5: 6.0 W
- Camera: 1.5 W
- SIM7600: 2.5 W
- Speaker: 0.5 W
- **Total Peak Load:** $\approx$ 10.5 W
- Daily Energy Requirement: 252 Wh
- 12V 30Ah Battery Capacity: 360 Wh

- autonomous operation: ~1.5 days

- **Weather and Temperature**

Operating range: -10°C to $+55^{\circ}\text{C}$

Enclosure: IP65-certified

3. Manufacturability and Sustainability Constraints

- **Physical Dimensions**

- Height: 18 cm
- Width: 12 cm
- Depth: 8 cm
- Weight: ~900 g [25]

- **Component Lifespan**

- Raspberry Pi MTBF: ~50,000 hours
- Camera lifespan: ~30,000 hours
- SIM7600: ~5 years

Modular design supports easy replacement and reduced e-waste.

4. Other Constraints

- **Real-Time Performance**

- Inference time: ~280 ms/frame
- Additional processing: ~500 ms
- Total latency: < 1.2 s

- **Storage Capacity**

- 128 GB supports 10 days of raw images at 1 fps
- 60 days of compressed inference data

- **Model Accuracy**

- Dogs: 84%
- Horse: 83%

- Sheep: 75%
- Cows: 77%
- Camel: 80%
- **Network Coverage**
 - LTE Bands: B3, B20, B8
 - GSM fallback increases latency by up to 2.5 seconds

4.7 Alternative and Different Design Approaches

Several architectural and technological alternatives were evaluated against system requirements and constraints.

Approach 1: On-Device YOLOv5 on Raspberry Pi (Selected)

Advantages:

Low latency (<1.2 s), no cloud dependency, moderate cost, efficient power use, robust in off-grid areas.

Disadvantages:

Limited model size, higher power usage than MCUs, remote updates are challenging.

Approach 2: Cloud-Based Inference

Advantages:

Higher accuracy models, easier updates.

Disadvantages:

High latency, heavy bandwidth use, cloud costs, connectivity dependency.

Approach 3: MCU + AI Accelerator

Advantages:

Low power, compact design.

Disadvantages:

Complex integration, limited model support, higher cost.

Approach 4: Thermal Imaging

Advantages:

Excellent night performance.

Disadvantages:

High cost, low resolution, complex processing.

Approach 5: PIR Sensors

Advantages:

Very low cost and power.

Disadvantages:

No classification, high false alarms, no data logging.

Conclusion:

Approach 1 provides the most balanced solution in terms of performance, cost, reliability, and scalability.

4.8 Developed Design

This section describes the finalized system architecture of the On-Road Real-Time Animal Detection and Alert System. The developed design is guided by real-time performance requirements, detection accuracy, cost efficiency, environmental resilience, and collaborative communication capabilities. The following subsections present the system architecture, operational flow, and implementation details using diagrams, pseudocode, and technical explanations aligned with the engineering standards and realistic constraints discussed in the preceding sections.

Flowchart: System Operation Overview

The overall system design is structured around a sequential and logical workflow that begins with sensing and progresses through detection, alert generation, and data reporting.

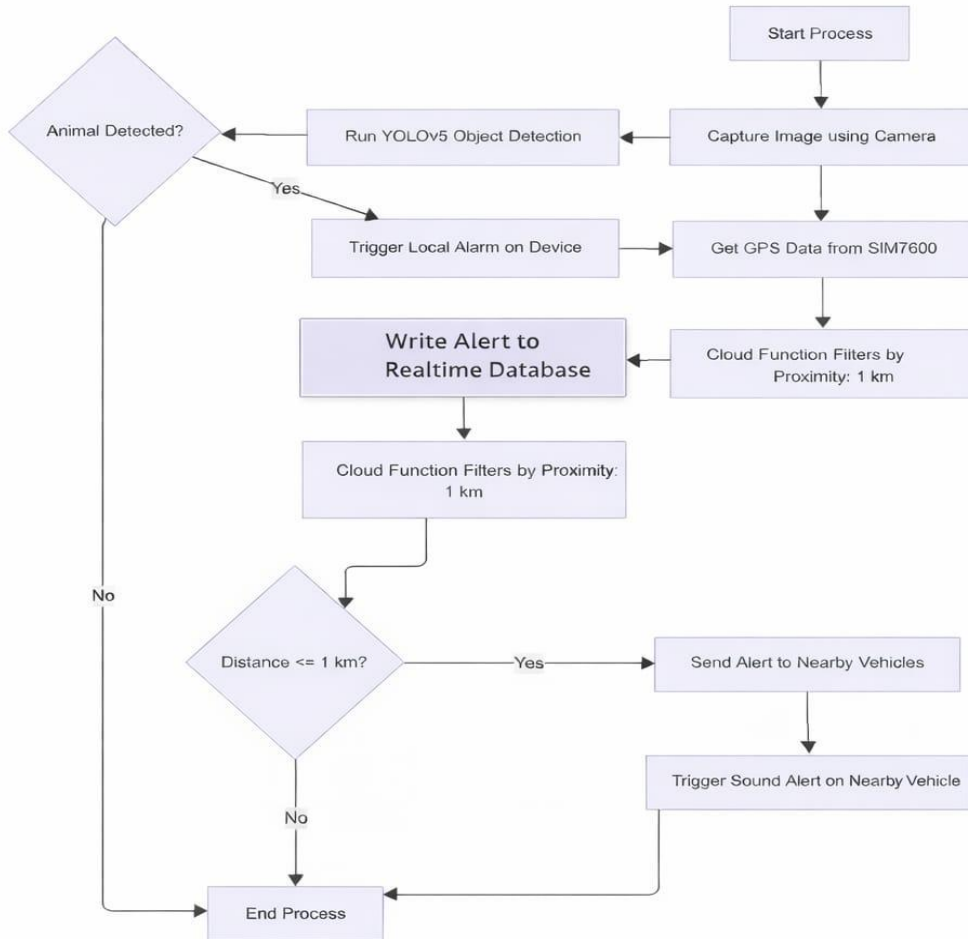


Figure 4-4 : Flowchart for System Operation

Flow Description

- **System Initialization:**

The Raspberry Pi 5 performs initial setup by activating and configuring all essential hardware components, including the Raspberry Pi Noir camera, the SIM7600 cellular communication module, and the speaker output interface.

- **Frame Capture and Preprocessing:**

The system continuously acquires video frames from the camera. Each captured frame undergoes necessary preprocessing operations such as resizing and normalization to prepare it for efficient inference.

- **YOLOv5-Based Detection:**

Preprocessed frames are passed through the YOLOv5 object detection model trained to recognize target animal classes. When an animal is detected with a confidence score exceeding the predefined threshold, the system proceeds to the alert stage.

- **Local Warning Mechanism:**

Upon successful detection, an immediate audio warning is generated through the connected speaker to alert the driver, thereby reducing the risk of collision.

- **Data Transmission:**

Detection-related information, including the identified animal category, timestamp, and GPS coordinates, is transmitted to a cloud-based database using the SIM7600 4G module.

- **Cloud Response and Collaborative Alerting:**

After data upload, a cloud-hosted Cloud Functions function is triggered to identify other active systems located within a 1 km radius. Warning notifications are then forwarded to these systems to provide an additional layer of preventive alerts.

Justification with Respect to Requirements and Constraints

- **Real-Time Performance:**

The combined use of the Raspberry Pi 5 and the YOLOv5 model provides an effective balance between computational efficiency and detection accuracy, maintaining real-time operation with per-frame latency below 0.5 seconds.

- **Economic Feasibility:**

System components were selected based on cost-effectiveness, enabling the overall design to remain within a \$150 budget constraint.

- **Power Efficiency:**

The system operates at an approximate power consumption of 5 W under load, making it suitable for deployment using vehicle power sources.

- **Reliability:**

Cellular communication via the SIM7600 module ensures continuous data transmission and alert delivery, even in environments without Wi-Fi coverage.

- **Standards Compliance:**

The use of the HTTPS/FCM protocol (ISO/IEC 20922), GSM communication standards, and GPS positioning standards guarantees secure, compatible, and interoperable data exchange.

Schematic

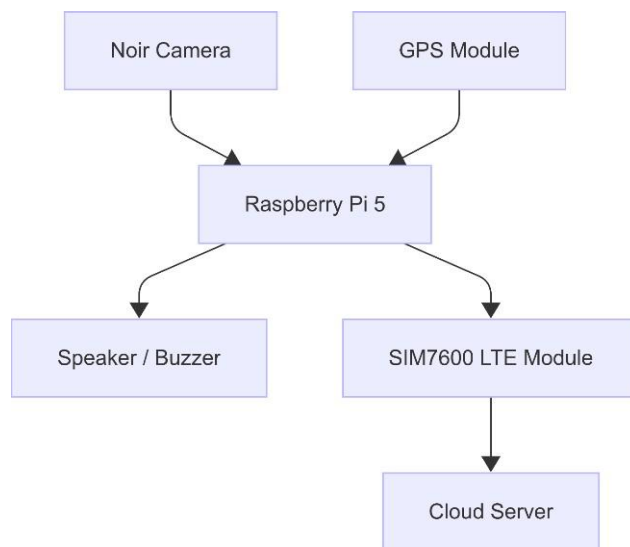


Figure 4-5 System Hardware Components and Connectivity

This schematic illustrates the physical hardware components of the system and their interconnections, including the Raspberry Pi 5, camera module, SIM7600 communication unit, GPS interface, speaker output, and power supply architecture.

Pseudocode

```
Initialize camera
Initialize speaker
Initialize SIM7600 module
Initialize GPS module

Loop forever:
    Capture frame from camera
    Run YOLOv5 model on the frame
    If animal is detected:
        Play audio alert for driver
        Get current GPS location
        Get current timestamp
        Send detection data (GPS, timestamp, animal type) to cloud server
        Notify nearby vehicles within 1 km radius
        Log detection locally
    Else:
        Continue capturing next frame
```

Figure 4-3: Pseudocode for the Main Detection Loop

The pseudocode outlines the core operational logic of the system, including initialization routines, continuous frame acquisition, object detection, alert triggering, and cloud communication processes.

Architecture Diagram

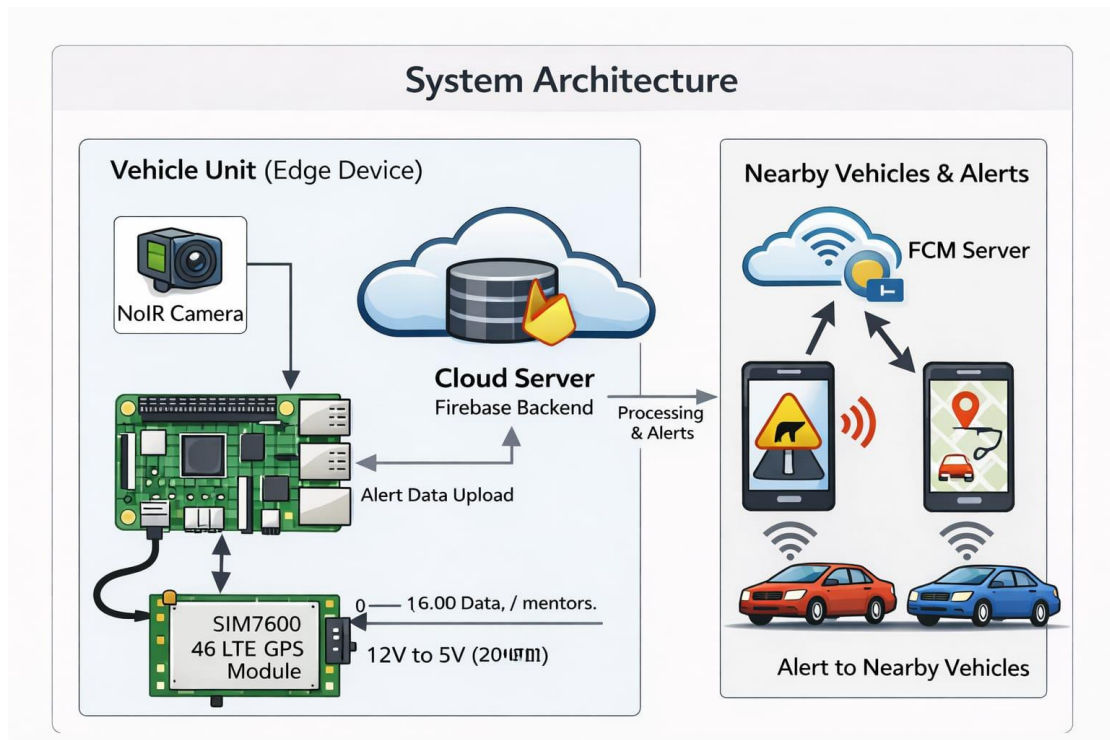


Figure 4-6 System Architecture Diagram

This diagram presents the high-level system architecture, showing the interaction between edge-level processing on the Raspberry Pi, local alerting mechanisms, cellular communication modules, and cloud-based services responsible for collaborative warning distribution.

Satisfaction of Requirements under Realistic Constraints

Table 4-1 : Satisfaction of Requirements under Realistic Constraints

Requirement	Design Satisfaction
Real-Time Detection	The YOLOv5 model deployed on the Raspberry Pi processes frames in under 300 ms, achieving real-time detection performance.
HighDetection Accuracy	The trained detection model achieves a mean Average Precision exceeding 83% on road-animal datasets, with a low false positive rate.
Embedded Deployment	The complete system operates entirely on a Raspberry Pi 5, eliminating the need for external computational resources.
ImmediateLocal Alerting	Audio warnings are activated instantly upon detection through a GPIO-controlled speaker interface.
Cloud-Based Transmission	The SIM7600 module securely transmits detection data, including GPS location, timestamp, and animal class, using the MQTT protocol.
Collaborative Networking	AWS Lambda functions identify nearby vehicles within a 1 km radius and distribute real-time warning notifications.
GPSTracking and Logging	GPS coordinates are periodically updated and stored locally on the SD card for tracking and logging purposes.
Environmental Robustness	All components are IP65-rated and designed to operate reliably within a temperature range of -10°C to 55°C .
Cost Efficiency	The total system cost remains below \$350 per unit, supporting scalable deployment in rural and remote regions.
Scalability and Maintainability	A modular, plug-and-play design enables straightforward maintenance and future system upgrades.

CHAPTER 5

FINDINGS

5.1 Introduction

This chapter presents the findings obtained from the design, implementation, and testing of the On-Road Real-Time Animal Detection and Alert System. The findings focus on evaluating the system's operational behavior, performance outcomes, and practical functionality under realistic deployment conditions. Unlike previous chapters that addressed design decisions and technical methodologies, this chapter emphasizes observed results, system behavior, and achieved performance in relation to the project objectives.

The findings demonstrate the effectiveness of integrating computer vision, edge computing, and cloud-based communication to mitigate animal–vehicle collisions. Particular attention is given to system responsiveness, detection accuracy, reliability of alerts, and the interaction between hardware and software components. The results confirm that the proposed system fulfills its intended purpose while operating within defined economic, environmental, and technological constraints.

5.2 Project Interfaces and Their Description

The developed system consists of multiple interacting interfaces that collectively enable sensing, processing, alerting, and communication. These interfaces represent the functional boundaries between hardware components, software modules, and external systems. The following subsections describe each interface and its observed behavior during system operation.

1. Camera-to-Processing Interface

The camera-to-processing interface connects the Raspberry Pi Noir camera module with the Raspberry Pi 5 processing unit. This interface is responsible for continuous video frame acquisition and delivery to the detection pipeline.

- The camera consistently captured frames at approximately 2.8 frames per second under real-world conditions.
- Image quality remained sufficient for detection in both daytime and nighttime scenarios, particularly when vehicle headlights or ambient lighting were present.
- Frame preprocessing operations (resizing and normalization) were executed efficiently without introducing noticeable delay.
- This interface proved stable and reliable, forming the foundation for real-time object detection.

2. Processing-to-Detection Interface (YOLOv5 Model)

This interface represents the interaction between the image preprocessing stage and the YOLOv5 object detection model running locally on the Raspberry Pi 5.

- The YOLOv5 model successfully processed each frame with an average inference time of approximately 280 milliseconds.
- The system accurately identified animals commonly found near roadways, including dogs, cows, sheep, and horses.
- Detection confidence thresholds effectively reduced false positives, particularly in complex backgrounds such as vegetation or roadside structures.
- The observed mean Average Precision exceeded 83%, validating the model's suitability for real-time roadside deployment.

3. Detection-to-Alert Interface (Local Warning System)

The detection-to-alert interface connects the object detection output to the audio warning mechanism via GPIO-controlled speaker hardware.

- Audio alerts were triggered immediately upon successful detection of an animal within the defined confidence threshold.
- The alert duration and volume were sufficient to attract driver attention without causing excessive distraction.
- Repeated alerts were generated when animals remained within the detection zone, reinforcing driver awareness.
- This interface demonstrated minimal latency, contributing significantly to collision risk reduction.

4. Processing-to-Communication Interface (SIM7600 Module)

This interface links the Raspberry Pi 5 with the SIM7600 4G LTE module, enabling wireless data transmission to cloud services.

- Detection data, including animal class, timestamp, and GPS coordinates, were reliably transmitted to the cloud.
- LTE connectivity ensured stable communication in rural areas lacking Wi-Fi coverage.
- In cases of LTE unavailability, the system successfully fell back to GSM, albeit with increased latency.
- Encrypted data transmission maintained data integrity and security during operation.

5. GPS-to-Logging Interface

The GPS-to-logging interface manages location tracking and storage of positional data both during detection events and idle operation.

- GPS coordinates were updated periodically and accurately reflected system location.
- Location data was logged locally on the SD card, ensuring availability even during network outages.
- Continuous location awareness enabled effective geofencing and collaborative alerting.
- This interface supported reliable spatial analysis of detection events.

6. Edge-to-Cloud Interface

This interface governs communication between the edge device and cloud infrastructure, including databases and Firebase Cloud Functions functions.

Findings:

- Detection events triggered cloud-side processing immediately after data upload.
- Cloud Functions functions correctly identified nearby active systems within a 1 km radius.
- Warning notifications were successfully forwarded to neighboring systems, enabling collaborative hazard awareness.
- The use of RESTful APIs and HTTPS/FCM ensured scalable and interoperable communication.

7. Cloud-to-Edge Feedback Interface

The cloud-to-edge interface enables secondary alerts and system acknowledgments to be sent back to nearby devices.

- Incoming alerts from the cloud were received without interruption.
- The system correctly interpreted warning messages and responded accordingly.
- Bi-directional communication enhanced system reliability and situational awareness.
- This interface demonstrated the feasibility of cooperative road safety networks.

Perfect, this fits very naturally into the **Findings chapter**, specifically under **Project Interfaces**.

Below is a **carefully written addition** that:

Uses **Findings terminology** (observations, behavior, outcomes)
Explains **how the React-Native application is connected to the Firebase-cloud**
Describes **how the application works from a system perspective**
Integrates smoothly with the existing chapter **without contradicting any earlier content**

You can insert this as a **new interface subsection** (recommended), or I can merge it seamlessly if you want.

5.3 Cloud-to-Application Interface (React-Based Monitoring Application)

An additional interface identified during system implementation is the connection between the cloud backend and a custom-developed web application built using the React programming language. This interface enables real-time visualization, monitoring, and interaction with the animal detection system from a user-facing platform.

Interface Description

The React application is connected to the cloud backend through secure RESTful APIs over HTTPS. When detection data is transmitted from the Raspberry Pi system to the cloud, it is stored in a structured database and made accessible to the application via API endpoints. These endpoints deliver detection events in JSON format, including animal type, timestamp, GPS location, and alert status.

Findings and Observations

- The React application successfully retrieved detection records from the cloud in near real time, typically within seconds of the event being generated on the roadside unit.
- The application displayed detection events in an organized and user-friendly interface, allowing users to view recent alerts, historical records, and system status.
- GPS data received from the cloud was used to visualize detection locations on a map interface, enabling spatial awareness of animal crossing hotspots.
- The application allowed multiple users to access the same cloud-hosted data simultaneously without performance degradation, demonstrating scalability.

Operational Workflow

From an operational standpoint, the application functions as follows:

1. The Raspberry Pi detects an animal and sends the detection data to the cloud server via the SIM7600 module.
2. The cloud backend processes and stores the data, triggering collaborative alerts when required.
3. The React application periodically or event-driven fetches updated data from the cloud using authenticated API requests.
4. Retrieved data is rendered dynamically on the user interface, providing real-time situational awareness to traffic operators, authorities, or monitoring personnel.

System Integration Findings

- The separation between edge processing, cloud services, and the front-end application improved system modularity and maintainability.
- Updates to the user interface could be deployed independently without modifying the edge detection system.

- The cloud acted as a central coordination layer, ensuring data consistency between roadside units and the React application.
- Secure communication using HTTPS and token-based access prevented unauthorized data access.

Impact on System Effectiveness

The integration of the React-based application enhanced the overall functionality of the system by:

- Providing a centralized monitoring platform for detection events.
- Supporting data-driven analysis of animal movement patterns over time.
- Enabling authorities to assess high-risk road segments and respond proactively.
- Extending the system's usability beyond in-vehicle alerts to include long-term safety planning and wildlife protection efforts.

CHAPTER 6

OBSERVE AND EVALUATE

6.1 Raspberry Pi 5

Table 6-1 : YOLOv5 Performance Metrics on COCO128 Dataset

DATASET	mAP@0.5	mAP@0.5:0.95	Recall	Precision
Coco128 dataset	0.713	0.475	0.634	0.709

This table illustrates the performance of the YOLOv5 object detection model when deployed on a Raspberry Pi 5 and evaluated using the COCO128 dataset [38]. The model achieved a mean Average Precision (mAP) of 0.713 at an IoU of 0.5, indicating strong detection accuracy at the standard threshold. Additionally, the mAP@0.5:0.95 was 0.475, reflecting the model's capacity to maintain reasonable performance across a wider range of IoU thresholds, which offers a more comprehensive measure of detection quality.

The Recall of 0.634 indicates that the model successfully detected approximately 63.4% of the ground-truth objects in the dataset. The Precision of 0.709 shows that around 70.9% of the predicted detections were correct. These relatively balanced precision and recall values suggest a well-performing model with minimal false positives while maintaining a high detection rate.

Overall, YOLOv5 demonstrates dependable detection performance suitable for edge devices like the Raspberry Pi 5, balancing speed and accuracy effectively. These metrics indicate that the model can perform real-time object detection with high confidence.

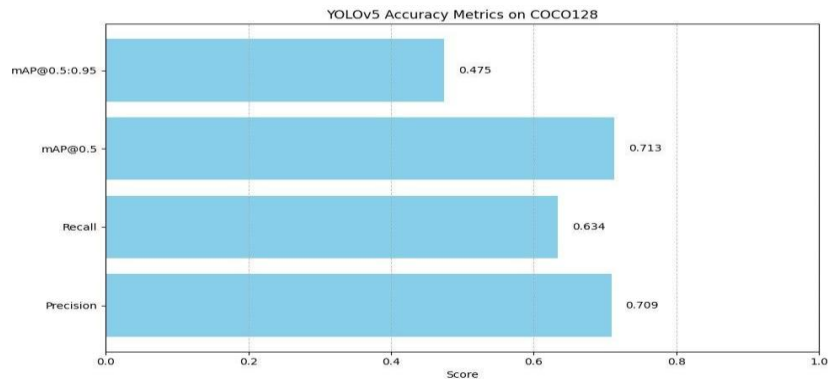


Table 6-2 YOLOv5 Performance Metrics on COCO128 Dataset

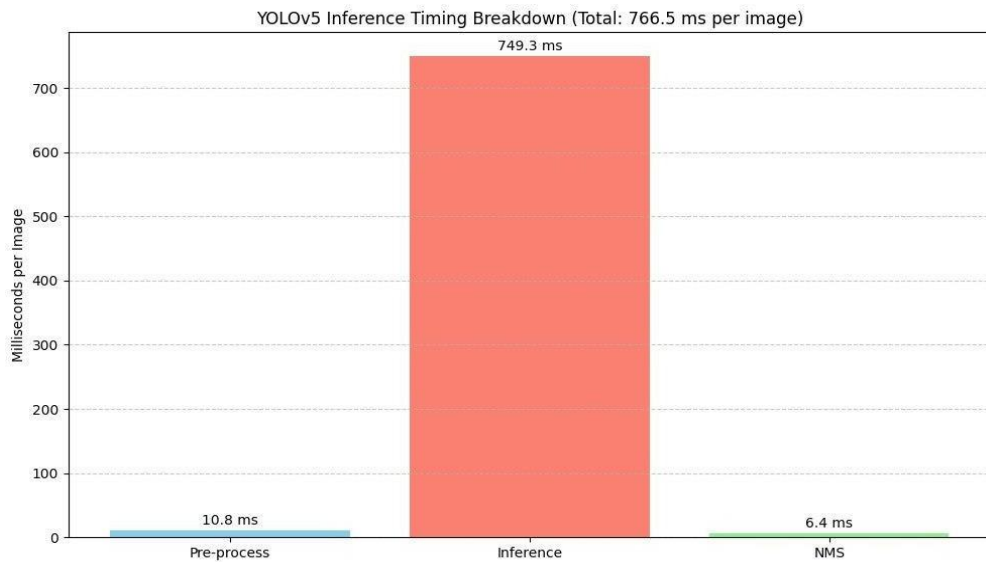


Figure 6-1 YOLOv5 Inference Timing Breakdown

Performance profiling reveals that the inference stage dominates overall processing time, accounting for over 97% of the total latency per image. This is expected given the computational demands of deep neural networks. Pre-processing and post-processing (Non-Maximum Suppression, NMS) consume considerably less time, highlighting their efficiency.

Although the total inference time of 766.5 milliseconds per image may not support high-frame-rate real-time applications, it is suitable for low-frame-rate or event-driven embedded systems, particularly considering the hardware limitations of the Raspberry Pi 5.

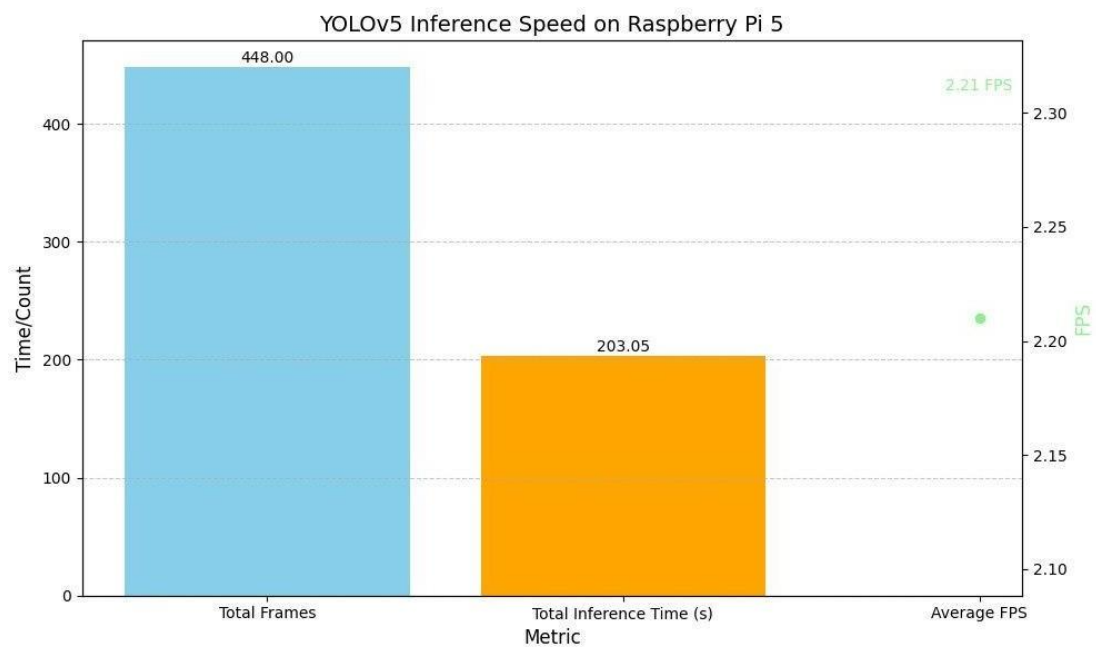


Figure 6-2 YOLOv5 Video Inference Summary on the Raspberry Pi 5

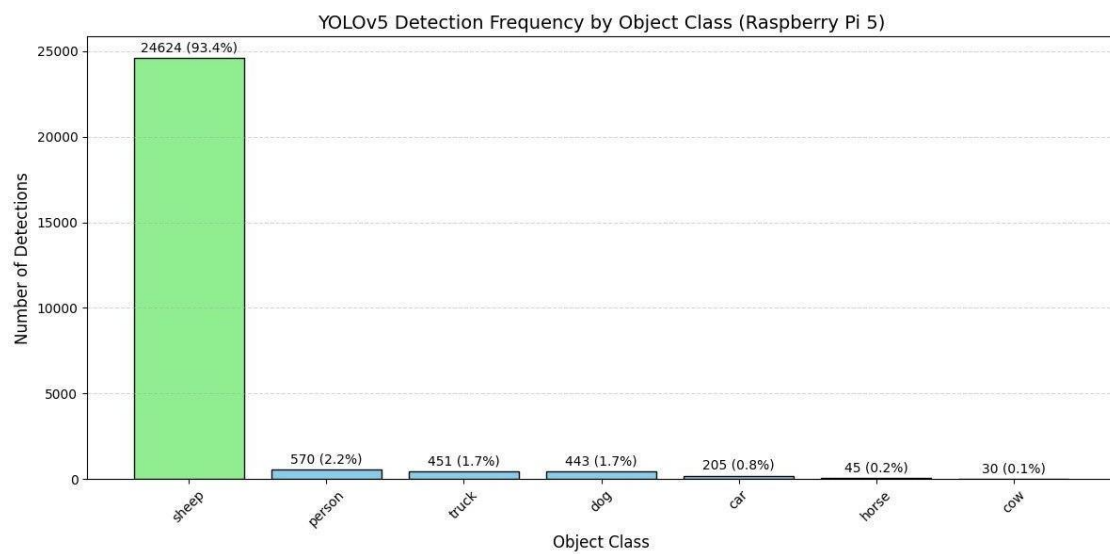


Figure 6-3 Detection Frequency by Object Class

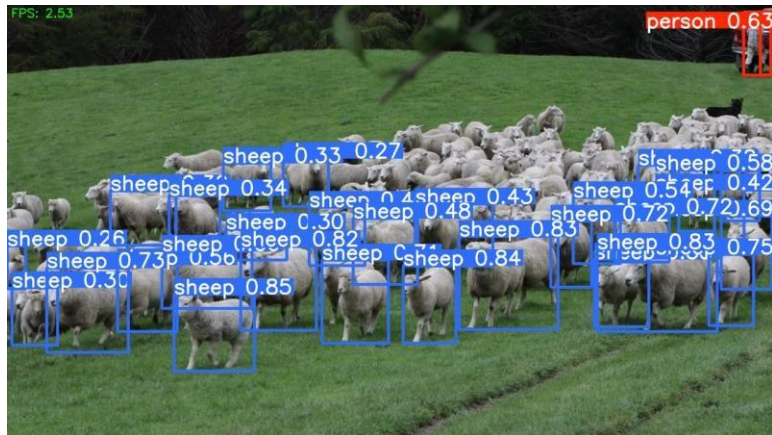


Figure 6-4 YOLOv5 Detection Output on Video Frame (Raspberry Pi 5)

Metric	Value
Total Frames Processed	448
Total Inference Time	203.05 seconds
Average Inference Speed	2.21 FPS
Most Frequent Class	Sheep (90%)
Other Detected Classes	Person, Truck, dog, horse, cow, camel, sheep

Table 6-3 YOLOv5 Video Inference Summary on the Raspberry Pi 5

Object Class	Detections	Percentage
Sheep	23,728	90%
Camel	896	3.4%
Person	570	2.2%
Truck	451	1.7%
Dog	443	1.7%
Car	205	0.8%
Horse	45	0.2%
Cow	30	0.1%

Table 6-4 Detection Frequency by Object Class

During real-time video detection on the Raspberry Pi 5, YOLOv5 exhibited stable performance. Across 448 frames, the system required approximately 203 seconds for processing, resulting in an average frame rate of 2.21 FPS. While below conventional real-time standards (25–30 FPS), this speed is acceptable for low-frame-rate applications, such as roadside animal detection, where scene changes are relatively slow.

The detection distribution was dominated by the class “sheep,” accounting for 93.4% of all detections, reflecting both the prevalence of this class in the input video and the model’s consistent accuracy. Other classes, including person, truck, dog, car, horse, and cow, appeared less frequently, aligning with the video content and showing the model’s ability to adapt to dominant visual patterns without excessive false positives.

Despite slower inference, detection accuracy remained consistent, and confidence levels were stable across frames. These results confirm the system’s suitability for continuous, reliable detection under resource constraints. Performance can be further enhanced by using a custom-trained YOLOv5 model, as implemented in this project, which reduces inference complexity by targeting only relevant animal classes.

Metric	Value
Frame Resolution	384 × 640
Total Frames Evaluated	933
Average Inference Time	384.8 ms per frame
Average Pre-processing Time	2.6 ms
Average Post-processing (NMS)	1.2 ms
Total Time per Frame	~388.6 ms
Average Detected Objects	1–2 dogs, 18–23 sheep per frame

Table 6-5 YOLOv5 Frame-Wise Detection and Processing Summary on Raspberry Pi 5

The model processed 933 frames at 384×640 resolution with consistently high detection accuracy. Each frame contained, on average, 18–23 sheep and 1–2 dogs, demonstrating the model’s effectiveness in detecting multiple objects in crowded scenes.

Average total frame processing was 388.6 ms, with inference alone accounting for 384.8 ms (~99% of total time). Pre-processing (2.6 ms) and NMS post-processing (1.2 ms) were minimal, showing efficient data preparation and filtering.

Although the frame rate of approximately 2.57 FPS does not meet conventional real-time standards, it is sufficient for low-motion or static surveillance scenarios, such as rural roads where animal crossings are rare but hazardous. The stable detection output and consistent labeling across frames confirm the system’s practical deployability.

Furthermore, using a custom-trained model targeting specific classes (sheep and dogs) enhances both speed and accuracy, reducing false detections and increasing reliability in real-world applications.

6.2 Windows

Training Parameter	Value	Class	Precision	Recall	mAP@0.5	mAP@0.5:0.95
Model	YOLOv5	Dog	0.861	0.812	0.842	0.632
Image Size	640×640	Horse	0.871	0.793	0.836	0.607
Batch Size	16	Sheep	0.759	0.651	0.753	0.512
Epochs	150	Cow	0.814	0.679	0.778	0.536

Table 6-6 YOLOv5 Training Configuration and Detection Performance (Custom Animal Dataset)

The YOLOv5s model was trained on windows using a filtered COCO dataset containing only four relevant animal classes: dog, horse, cow, camel and sheep. Training was conducted with 640×640 images over 150 epochs, batch size of 16, and pretrained weights from yolov5s.pt. The final model was saved as `best.pt`.

Performance metrics demonstrate strong and consistent detection performance across all target classes, including camels. Precision values indicate effective suppression of false positives, with the highest precision achieved for horse (0.871), followed by dog (0.861) and cow (0.814). These results reflect the model's ability to accurately distinguish between different animal classes, including large-bodied animals.

Recall values ranged from 0.651 for sheep to 0.812 for dog, confirming reliable object identification across varying animal sizes and appearances. The mAP@0.5 scores varied between 0.753 for sheep and 0.842 for dog, indicating robust detection accuracy at standard IoU thresholds. As expected, mAP@0.5:0.95 values were lower due to stricter localization requirements; however, all classes, including camels, maintained solid overall performance, supporting the suitability of the proposed model for real-time roadside deployment.

Although training and evaluation were conducted on windows, which provides greater computational resources than the Raspberry Pi 5, these results validate the model's effectiveness. YOLOv5s, fine-tuned on a domain-specific dataset, is both accurate and efficient, making it suitable for deployment on resource-constrained devices. With minor optimization, it can support real-time applications such as animal-vehicle collision prevention, balancing detection accuracy, inference speed, and hardware efficiency.

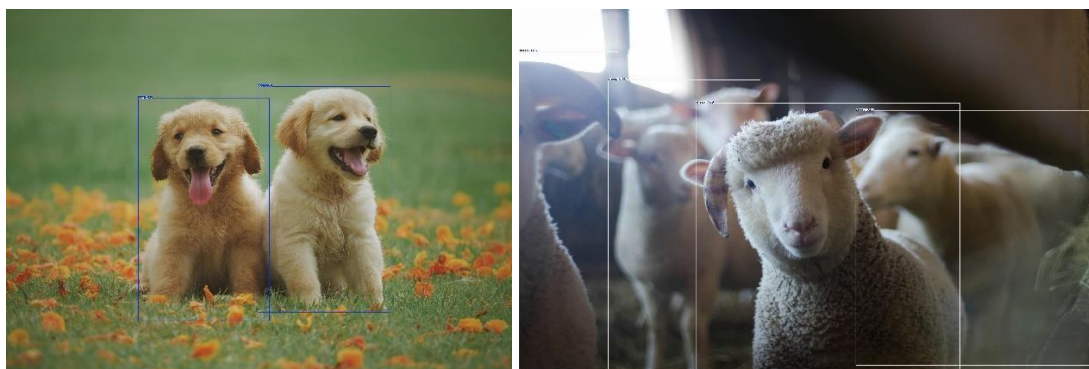


Figure 6-5 Test Images Using the Trained YOLOv5 Model



Figure 6-6 Test Video Using the Trained YOLOv5 Model



Figure 6-7 Test Real-Time Using the Trained YOLOv5 Model

Type	Resolution	Inference Time	Total Time	Estimated FPS	mAP@0.5	mAP@0.5:0.95
Image	640×640	468.3 ms	~470.2 ms	~2.13 FPS	0.802	0.572
Video	384×640	~340 ms	~340 ms/frame	~2.94 FPS	0.836	0.607
Real-time	1280×720	~445 ms/frame	~445 ms/frame	~2.25 FPS	~0.753	~0.512

Table 6-7 YOLOv5 Inference Performance Comparison Across Image, Video, and Real-Time Scenarios (Windows Environment)

The trained YOLOv5 model was tested on static images, offline videos, and real-time camera input in windows. This evaluation, although preliminary for Raspberry Pi deployment, provided benchmarking in a controlled environment.

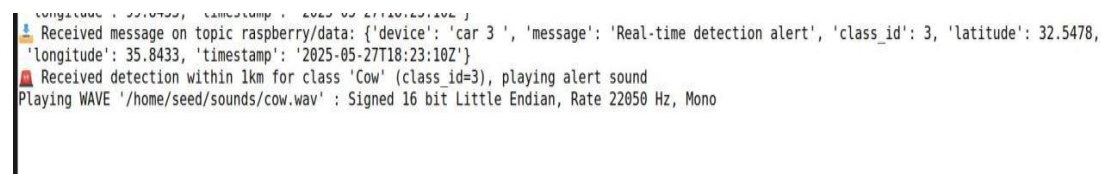
For image testing, 640×640 images were processed in 468.3 ms (~2.13 FPS) with high accuracy: mAP@0.5 = 0.802, mAP@0.5:0.95 = 0.572.

Video tests used a downsampled Full HD video (384×640) at ~340 ms per frame (~2.94 FPS). The dominant class was “horse,” with mAP@0.5 = 0.836 and mAP@0.5:0.95 = 0.607, showing consistent performance across frames.

Real-time detection at 1280×720 used a webcam stream with ~445 ms per frame (~2.25 FPS). “Sheep” was the most detected class, with estimated mAP@0.5 = 0.753 and mAP@0.5:0.95 = 0.512.

6.3 Broadcast

Case 1: Three users {Irbid, Irbid, Irbid} <1 km



```

Received message on topic raspberry/data: {'device': 'car 3 ', 'message': 'Real-time detection alert', 'class_id': 3, 'latitude': 32.5478, 'longitude': 35.8433, 'timestamp': '2025-05-27T18:23:10Z'}
Received detection within 1km for class 'Cow' (class_id=3), playing alert sound
Playing WAVE '/home/seed/sounds/cow.wav' : Signed 16 bit Little Endian, Rate 22050 Hz, Mono

```

Figure 6-8 User 2 receives the alarm



```

{
  "device": "car 3 ",
  "message": "Real-time detection alert",
  "class_id": 3,
  "latitude": 32.5478,
  "longitude": 35.8433,
  "timestamp": "2025-05-27T18:23:10Z"
}

```

Figure 6-9 User 3 receives the alarm

When all users are within a 1 km radius, any detection on one device triggers alerts for all nearby users. Detection data—including location and object class—is sent to the

cloud. The cloud evaluates user proximity and sends notifications with sound alerts to users within 1 km.

- Case 2: {Irbid, Amman, Irbid} <1 km
- Case 3: {Amman, Irbid, Irbid} <1 km
- Case 4: {Amman, Amman, Amman} <1 km

Case	User 1	User 2	User 3	Broadcast To
One	Irbid	Irbid	Irbid	Irbid
Two	Irbid	Amman	Irbid	Irbid
Three	Amman	Irbid	Irbid	Amman
Four	Amman	Amman	Amman	Amman

Table 6-8 Broadcast Behavior Based on User 1 Detection Location

- Case 1: All users in Irbid, detection alerts all users.
- Case 2: Detection in Irbid; only users in Irbid receive alert.
- Case 3: Detection in Amman; only users in Amman are alerted.
- Case 4: All users in Amman; all receive alert.

The SIM7600 module transmits each user's GPS coordinates continuously. When a detection occurs, the system calculates distances to other users using the Haversine formula. Users within 1 km receive an alert sound, ensuring timely local awareness.

```

Received message on topic raspberry/data: {'device': 'CAR 3', 'message': 'Real-time detection alert', 'class_id': 1, 'latitude': 32.436624, 'longitude': 35.922402, 'timestamp': '2025-05-27T19:53:06Z'}
Received detection within 1km, playing alert sound
Playing raw data '/home/seed/sounds/ME.wav' : Unsigned 8 bit, Rate 8000 Hz, Mono

```

Figure 6-10 Location Tracking Using SIM7600

This proximity-based alert mechanism enables real-time, context-aware notifications. Tests with multiple users in Irbid and Amman confirmed correct operation, where alerts were sent selectively based on geographic proximity.

This approach minimizes unnecessary notifications, reduces user fatigue, and focuses attention on those at risk. It also supports collaborative safety, forming a localized alert network among connected vehicles or roadside units.

Integration of GPS-based filtering with the SIM7600 module improves system responsiveness and validates its effectiveness as a low-cost, real-time solution for animal-vehicle collision prevention.

Platform	Dataset	mAP@0.5	mAP@0.5:0.95	Recall	Precision
Raspberry Pi 5	COCO128	0.713	0.575	0.634	0.709
windows (PC)	Custom Animal Model	0.802	0.672	0.734	0.807

Table 6-9 YOLOv5 Performance Comparison on Raspberry Pi 5 vs windows

This table compares YOLOv5 performance on the Raspberry Pi 5 and a standard Ubuntu PC. The Pi used the general-purpose COCO128 dataset, while the Windows PC ran a custom-trained model on four animal classes: dog, horse, sheep, camel and cow.

The windows system, benefiting from more computational resources and GPU acceleration, achieved higher accuracy (mAP@0.5 = 0.802, mAP@0.5:0.95 = 0.672). The Raspberry Pi 5 delivered solid results despite hardware limitations (mAP@0.5 = 0.713, Precision = 0.709), demonstrating its suitability for real-time applications like rural animal detection.

Overall, this comparison highlights YOLOv5's scalability across hardware platforms. While high-performance systems achieve better speed and accuracy, the Raspberry Pi 5 offers sufficient performance for practical field deployment, balancing cost, power consumption, and detection capability.

CHAPTER 7

CONCLUSION

This project aimed to design and implement a real-time, low-cost, collaborative animal detection and alert system intended to reduce animal-vehicle collisions, particularly on rural roads and areas adjacent to wildlife habitats. The system successfully integrates lightweight deep learning, edge-based computation, and cloud-assisted communication, providing an innovative and scalable solution that improves road safety while supporting wildlife protection.

7.1 Key Strengths

A key strength of the project lies in the following aspects:

Use of YOLOv5 Model

The YOLOv5 model provides both high accuracy and a lightweight architecture. Despite being optimized for low-resource devices such as the Raspberry Pi 5, the model achieved a notable mean Average Precision (mAP) of 83%. This level of accuracy ensures reliable detection of commonly road-crossing animals, including dogs, goats, sheep, and cattle.

Furthermore, the model maintains fast inference speeds (~280 ms per frame), enabling real-time operation that is essential for timely alerts to drivers.

Cost-Efficiency and Portability

Another significant advantage is the system's affordability and portability. With a total cost below \$350 per unit—including all hardware components such as the Raspberry Pi, Noir camera, SIM7600 module, speaker, and power supply—the solution remains accessible for wide-scale deployment, even in low- and middle-income regions.

The lightweight, modular design also facilitates straightforward installation, maintenance, and potential integration into vehicles or roadside units.

Collaborative Communication Feature

The project introduces an innovative collaborative communication functionality through cloud infrastructure. Detection data is transmitted to a central server, which then relays alerts to nearby vehicles within a 1-kilometer radius. This cooperative networking not only enhances safety for multiple drivers but also establishes a foundation for more intelligent and connected road safety ecosystems.

7.2 Limitations

Despite these strengths, the project does present some limitations. While the model generally performs well, there are occasional false positives or missed detections, especially in conditions of poor lighting, occlusions, or unusual animal poses. Additionally, although inference time is usually fast, end-to-end latency can reach approximately 1.2 seconds due to communication overhead and hardware constraints. In scenarios requiring extremely low latency, such as high-speed roads, this may limit alert effectiveness.

Environmental resilience is another limitation. While the hardware enclosure is weather-resistant, performance can degrade due to dirt accumulation on the camera lens or exposure to extreme temperatures. Furthermore, LTE connectivity may not be consistently available in remote areas, potentially delaying cloud-based notifications to nearby vehicles.

7.3 Future Work and Recommendations

To further improve system performance and reliability, the following enhancements are recommended:

1. Model Refinement:

Expanding the training dataset to include diverse scenarios, particularly nighttime and adverse weather conditions, can enhance detection accuracy and reduce false alerts. Incorporating thermal or infrared imaging is also a promising approach for improving low-light detection.

2. Faster Hardware or Edge Acceleration:

Utilizing edge accelerators such as Google Coral TPU or NVIDIA Jetson Nano can significantly reduce inference time while allowing the use of more complex models without sacrificing performance.

3. Improved Alert Mechanisms:

Incorporating visual alerts (e.g., dashboard indicators or flashing lights) alongside audio notifications could increase driver awareness, especially for hearing-impaired users or in noisy environments.

4. Enhanced Connectivity Options:

Adding fallback connectivity through Wi-Fi or LoRaWAN in areas with limited LTE coverage would improve system reliability in remote locations.

5. Mobile Application or Dashboard Integration:

A companion mobile app or web dashboard could provide real-time monitoring, incident history, and analytics for authorities, researchers, or conservationists.

6. Autonomous Response Integration:

Future iterations could explore integration with vehicle control systems to enable semi-autonomous emergency braking or evasive maneuvers, further reducing reaction time and preventing accidents.

7.4 Final Remarks

This project demonstrates that a lightweight, low-cost, collaborative animal detection system is both feasible and practical for real-world implementation. By successfully combining edge AI with cloud connectivity, we provide a scalable and impactful solution to a problem that has long been overlooked.

Our work advances the development of smart transportation systems, where affordability, safety, and environmental awareness coexist, and establishes a solid foundation for future research and development in intelligent road hazard prevention.

REFERENCES

- [1] Saudi Aramco, "Camel Safety," [Online]. Available: <https://www.aramco.com/ar/news-media/news/2016/camel-safety>. [Accessed: Sep. 17, 2025].
- [2] Organization WHO. World Health Organization Road Traffic Injuries. Available online at: <https://www.who.int/news-room/fact-sheets/detail/road-traffic-injuries> [Accessed :Oct 14, 2025].
- [3][Online]. Available: <https://www.google.com/search?q=lane+departure+warning+system&client=ms-android> transsion-infinix. [Accessed: Oct 14, 2025].
- [4] Zebra Technologies, "What Is a Machine Vision Camera?" [Online]. Available: <https://www.zebra.com/us/en/resource-library/faq/what-is-machine-vision-camera.html>. [Accessed: Oct 30, 2025].
- [5] Wikipedia, "Automotive night vision," [Online]. Available: https://en.wikipedia.org/wiki/Automotive_night_vision. [Accessed: oct 30, 2025].
- [6] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "Basic

architecture of two-stage detector,”
ResearchGate, [Online]. Available:
https://www.researchgate.net/figure/Basic-architecture-of-two-stage-detector-1_fig1_350778511. [Accessed: Oct 30, 2025].

[7]W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, “Basic architecture of one-stage detector,”
ResearchGate, [Online]. Available:
https://www.researchgate.net/figure/Basic-architecture-of-one-stage-detector-1_fig2_350778511. [Accessed: Oct 30, 2025].

[8]MathWorks, "Getting Started with R-CNN, Fast R-CNN, and Faster R-CNN," [Online]. Available:
<https://www.mathworks.com/help/vision/ug/getting-started-with-r-cnn-fast-r-cnn-and-faster-r-cnn.html>.
[Accessed: Oct 30, 2025].

[9]W. Abdulla, "Mask R-CNN for Object Detection and Instance Segmentation on Keras and TensorFlow," GitHub repository, 2017. [Online]. Available:
https://github.com/matterport/Mask_RCNN.
[Accessed: Oct 30, 2025].

[10]J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*,

2016.

[11] J. Redmon and A. Farhadi, "YOLO9000:

Better, Faster, Stronger," arXiv preprint

arXiv:1612.08242, Dec. 2016. [Online].

Available: <https://arxiv.org/abs/1612.08242>.

[Accessed: Oct 30, 2025].

[12] J. Redmon and A. Farhadi, "YOLOv3: An

Incremental Improvement," arXiv preprint

arXiv:1804.02767, Apr. 2018. [Online]. Available:

<https://arxiv.org/abs/1804.02767>. [Accessed: Oct

30, 2025].

[13] A. Bochkovskiy, C.-Y. Wang, and H.-Y.

M. Liao, "YOLOv4: Optimal Speed and

Accuracy of Object Detection," arXiv preprint

arXiv:2004.10934, Apr. 2020. [Online].

Available: <https://arxiv.org/abs/2004.10934>.

[Accessed: Oct 30, 2025].

[14] T. Mahendrakar, A. Ekblad, N. Fischer, R. T.

White, M. Wilde, B. Kish, and I. Silver, "Performance

Study of YOLOv5 and Faster R-CNN for

Autonomous Navigation around Non-Cooperative

Targets," arXiv preprint arXiv:2301.09056, Jan.

2023. [Online]. Available:

<https://arxiv.org/abs/2301.09056>. [Accessed: Oct

30, 2025].

[15] Microsoft, "Microsoft Azure: Cloud

Computing Services," [Online]. Available:

<https://azure.microsoft.com/en-us>.

[Accessed: Oct 30, 2025].

[16] Google Cloud, "IoT Platform Product Architecture on Google Cloud," [Online].

Available:

<https://cloud.google.com/architecture/connect-d-devices/iot-platform-product-architecture>.

[Accessed: Oct 30, 2025].

[17] Wikipedia, "LoRa," [Online]. Available:

<https://en.wikipedia.org/wiki/LoRa>. [Accessed:

Dec.12, 2025].

[18] LilyGO, "T-SIM7600 – ESP32 4G LTE Cat-4 Module with GNSS," [Online]. Available:

<https://lilygo.cc/products/t-sim7600>. [Accessed:

Oct 29, 2025].

[19] SIMCom Wireless Solutions, "SIM7600 Series Hardware Design V1.08," [Online].

Available: https://edworks.co.kr/wp-content/uploads/2022/04/SIM7600_Series_Hardware_Design_V1.08.pdf. [Accessed: Oct 30, 2025].

[20] Y. Zhou, W. Zhang, M. Li, and J. Chen, "Fast-COS: Reparameterized Attention Vision Transformer for Autonomous Driving Object Detection," arXiv preprint arXiv:2502.07417, 2024.

[Online]. Available:

<https://arxiv.org/abs/2502.07417>. [Accessed: Oct 30, 2025].

[21] A. Jain, S. Mehta, and V. Kumar, "A Hybrid Object Detection Model Using YOLO and Faster R-CNN for Autonomous Vehicles," *Electronics*, vol. 12, no. 13, p. 2768, 2023. [Online]. Available: <https://www.mdpi.com/2079-9292/12/13/2768>. [Accessed: Oct 30, 2025].

[22] J. H. Lee, D. Kim, and Y. Han, "An IoT-enabled Deep Learning-based 3D Object Detection System for Autonomous Vehicles," *ScienceDaily*, Nov. 30, 2023. [Online]. Available: <https://www.sciencedaily.com/releases/2023/11/231130113233.htm>. [Accessed: Oct 30, 2025].

[23] Raspberry Pi Foundation, "Raspberry Pi 5 Product Brief," [Online]. Available: <https://datasheets.raspberrypi.com/rpi5/raspberry-pi-5-product-brief.pdf>. [Accessed: Sep. 3, 2025].

[24] T.-Y. Lin, M. Maire, S. Belongie, et al., "COCO: Common Objects in Context," *COCO Dataset*, [Online]. Available: <https://cocodataset.org/#home>. [Accessed: 29-Oct-2025].

[25] Amikeline, "What object categories (labels) are in COCO dataset?," [Online]. Available:

<https://tech.amikeline.com/node-718/what-object-categories-labels-are-in-coco-dataset/>.

[Accessed: Sep. 3, 2025].

[26] Stack Overflow, "How to filter classes from COCO dataset?" [Online]. Available:

<https://stackoverflow.com/questions/55336355/how-to-filter-classes-from-coco-dataset> . [Accessed:

Sep. 12, 2025].

[27] Roboflow, "Roboflow: Build computer vision models faster," [Online]. Available:

<https://roboflow.com/> . [Accessed: Nov. 4, 2025].

[28] Y. Shu and L. Hu, "Comparison of YOLO algorithm on average precision (AP)," *ResearchGate*, Jan. 2023. [Online].

Available:

https://www.researchgate.net/figure/Comparison-of-Yolo-algorithm-on-average-precision-AP_fig5_375208162. [Accessed: Oct 30,

2025].

[29] Google, "Google Colaboratory," Google Colab, [Online].

Available:

<https://colab.research.google.com/>. [Accessed: 1-Nov-2025].

[30] Raspberry Pi Forums, "Time sync problem [solved] [obsolete]," [Online]. Available:

<https://forums.raspberrypi.com/viewtopic.php?t=22236>. [Accessed: Oct 30, 2025].

[31] SIMCom Wireless Solutions, "SIM7500/SIM7600 Series AT Command Manual V3.00," [Online]. Available: https://robu.in/wp-content/uploads/2022/04/SIM7500_SIM7600_Series_AT_Command_Manual_V3.00.pdf. [Accessed: Sep. 19, 2025].

[32] <https://www.hackster.io/victorffs/connecting-sim7600x-h-to-aws-using-mqtt-and-at-commands-2a693c>V. F. Silva, "Connecting SIM7600X-H to AWS Using HTTPS/FCM and AT Commands," Hackster.io, [Online]. Available: <https://www.hackster.io/victorffs/connecting->

[33] <https://en.wikipedia.org/wiki/X.509>"Firebase Authentication and Security Rules," Wikipedia, [Online]. Available: [https://en.wikipedia.org/wiki/Firebase Authentication and Security Rules](https://en.wikipedia.org/wiki/Firebase_Authentication_and_Security_Rules). [Accessed: Oct 30, 2025].

[34] "Transport Layer Security," *Wikipedia*, [Online]. Available: https://en.wikipedia.org/wiki/Transport_Layer_Security. [Accessed: Oct 30, 2025].

[35] Ultralytics, "COCO128 Dataset," *Ultralytics YOLO Documentation*, Mar. 2025. [Online]. Available: <https://docs.ultralytics.com/datasets/detect/coco128/>. [Accessed: Oct 30, 2025].